TUM Data Innovation Lab

Munich Data Science Institute (MDSI)

Technical University of Munich

&

CV4DT Research chair, University of Cambridge

Final report of project:

Semantic Planar Splatting

Authors	Binyu Fang, Katy Grossmann, Billy Herrmann, Danit Niwattananan, Pablo Díaz Viñambres
Mentors	Prof. Dr. Olaf Wysocki, B.Sc. Jiayue Yang
TUM Mentors	Prof. Dr. Yan Xia, M.Sc. Qilin Zhang, M.Sc. Wejdene Mansour
Project lead	Dr. Ricardo Acevedo Cabra (MDSI)
Supervisor	Prof. Dr. Massimo Fornasier (MDSI)

Aug 2026

Acknowledgements

We would like to thank the CV4DT Research Chair at the University of Cambridge and Prof. Dr. Olaf Wysocki for proposing this project, for guidance on the problem domain, and for providing compute resources that made this work possible. We further thank M.Sc. Qilin Zhang and M.Sc. Wejdene Mansour for their support and valuable feedback during our weekly discussions. Finally, we thank the TUM-DI-LAB team and Dr. Ricardo Acevedo Cabra for organizing the lab and making this project possible.

Abstract

Novel view synthesis (NVS) and 3D scene reconstruction often face a fundamental trade-off between geometric interpretability, memory compactness, and photometric fidelity. While recent advances in 3D and 2D Gaussian Splatting provide high-quality, real-time rendering, they typically lack an explicit, structured understanding of the macroscopic scene geometry. These unstructured representations often generate millions of redundant primitives, struggling to recover clean, flat surfaces and frequently producing noisy geometry with floating artifacts.

To address these limitations, we propose a novel hybrid scene decomposition and rendering pipeline that models full complex scenes by explicitly exploiting the varying geometric complexity of different regions. Our method decomposes indoor scenes into two complementary parametric primitives: planes to robustly capture flat, structural backgrounds (e.g., walls, floors, and ceilings), and superquadrics to represent complex, curved, or irregular foreground objects (e.g., furniture and clutter).

The proposed pipeline operates in two progressive phases. First, an initialization and geometric optimization phase fits these explicit primitives utilizing multi-view segmentation masks, along with depth and normal estimates. This stage is constrained by a carefully designed joint loss combining photometric consistency with geometric regularizations such as primitive parsimony, spatial coverage, and normal alignment to ensure a globally consistent geometric scaffold. Second, a photometric optimization phase samples and initializes 2D Gaussian splats directly onto these primitive surfaces. While initially anchored to the geometric proxies, these splats are subsequently refined to capture high-frequency texture details and achieve photorealistic NVS without sacrificing the underlying structural integrity.

By grounding 2D splats on explicit geometric scaffolds, our approach achieves competitive rendering metrics (e.g., PSNR, SSIM, LPIPS) on standard benchmarks such as ScanNet++, while recovering highly accurate surface meshes and normal consistency using significantly fewer primitives than traditional methods.

Beyond its technical contributions, this work also brings about considerable social importance. By drastically reducing the memory footprint and computational requirements for storing and rendering high-fidelity 3D environments, our efficient scene representation will eliminate many obstacles concerning the extensive use of AR/VR applications that require heavy computing power and memory. Furthermore, the explicit decomposition of scenes into geometrically interpretable entities distinctly separates structural boundaries from interactive objects and directly facilitates critical real-world downstream tasks, such as safe autonomous robotic navigation, accessible smart home automation, and the scalable deployment of digital twins for sustainable architectural planning.

Contents

Abstract **2**

1 Introduction **5**

2 Related Work **7**
2.1 Novel View Synthesis . . . 7
2.2 Geometric Primitives for Reconstruction . . . 8
2.2.1 Volumetric Primitives . . . 8
2.2.2 Planar Primitives . . . 9
2.2.3 Hybrid Primitive Representations . . . 9
2.3 Monocular Depth Estimation . . . 10
2.4 Segmentation Model . . . 11

3 Method **12**
3.1 Overview of the Pipeline . . . 12
3.2 Point Cloud Initialization . . . 12
3.2.1 Motivation and Requirements . . . 12
3.2.2 MoGe Cloud Densification . . . 14
3.2.3 LaSSM 3D Instance Segmentation . . . 14
3.3 Scene Representation . . . 15
3.3.1 Geometric Representation . . . 15
3.3.2 2D Gaussian skins for Appearance Modeling . . . 17
3.3.3 Primitive Initialization . . . 18
3.4 Optimization framework . . . 19
3.4.1 Geometric phase . . . 19
3.4.2 Photometric phase . . . 21

4 Experiments **22**
4.1 Experimental Setup . . . 22
4.1.1 Datasets . . . 22
4.1.2 Baselines . . . 22
4.1.3 Training and Inference Configuration . . . 23
4.1.4 Evaluation Metrics . . . 24
4.2 Quantitative Results . . . 25
4.2.1 Geometry Reconstruction Performance . . . 25
4.2.2 Novel View Synthesis Performance . . . 25
4.2.3 Compactness Analysis . . . 26
4.3 Qualitative Results . . . 27

5 Discussion **30**
5.1 Preprocessing Challenges . . . 30
5.2 Challenges with Training . . . 30
5.3 Limitations . . . 30

6 Conclusion **31**

References **32**

Appendix **36**

1 Introduction

Reconstructing indoor scenes from multi-view images is a long-standing problem in computer vision, with applications in mixed reality, robotic navigation, and the digital twins used for building modeling. These applications require a scene representation that is at once photorealistic, geometrically accurate, and compact and interpretable, so that a person or an autonomous system can render, navigate, and act on the reconstruction. Monocular camera-based capture is preferred for this task, as it is lightweight and inexpensive compared to depth sensors and laser scanners.

Achieving these properties together is difficult, because indoor scenes are geometrically heterogeneous. Walls, floors, and ceilings occupy most of the surface area yet are simple planar structures, whereas the furniture and objects between them are small but carry most of the geometric complexity. Existing methods, having developed the two components of our goal, explicit decomposition and photorealistic rendering, along separate lines, are ill-matched to this structure. Radiance-field methods, led by 3D Gaussian Splatting [22], render such scenes photorealistically but represent them as unstructured clouds of millions of kernels, with no notion of the objects or surfaces they contain. Explicit primitive-based methods provide the converse, a compact and interpretable structure, but encode geometry without appearance and cannot synthesize images. Recent hybrid methods couple the two, yet stop short of full rooms: PartGS [15] assumes object-centered, background-free inputs and a single primitive family, while 3D Gaussian Flats [43] reaches indoor scenes but treats its planes as constraints on Gaussians rather than as primitives of an object-level decomposition.

We present Primimix, a hybrid scene decomposition and rendering pipeline that assigns each part of a scene the primitive matching its geometry: bounded planes for the structural surfaces of walls, floors, and ceilings, and continuous superquadrics for the foreground objects. Every primitive is made renderable by binding 2D Gaussian skins to its surface, so that a single representation carries both an interpretable geometric scaffold and photorealistic appearance, recovered from posed images alone.

Our contributions are summarized as follows:

- We propose a visual-based dual-primitive scene representation that models structural surfaces with bounded planes and foreground objects with superquadrics within a single explicit, interpretable decomposition.
- We demonstrate that binding 2D Gaussian skins to both plane and superquadric primitives in a joint geometric optimization yields complete, interpretable geometry for downstream tasks while retaining photorealistic novel view synthesis.
- We show on ScanNet++ [57] that Primimix attains rendering quality competitive with unstructured Gaussian baselines and accurate surface geometry, while describing each scene with orders of magnitude fewer explicit primitives.

This report is organized as follows. Sec. 2 reviews the work related to our method, covering novel view synthesis in Sec. 2.1, geometric primitives for reconstruction in Sec. 2.2, monocular depth estimation in Sec. 2.3, and segmentation models in Sec. 2.4. In Sec. 3,

we then present our method, starting with the image-based point-cloud initialization in Sec. 3.2, which densifies and segments the scene into primitive-ready instances. We then introduce our dual-primitive scene representation and its 2D Gaussian skinning in Sec. 3.3, and detail the two-phase optimization, geometric followed by photometric, in Sec. 3.4.1 and Sec. 3.4.2. The details regarding our experiments are then described in Sec. 4, starting with the dataset, baselines, and evaluation metrics in Sec. 4.1. We then report our quantitative results on geometry reconstruction, novel view synthesis, and compactness in Sec. 4.2.1 to Sec. 4.2.3, followed by qualitative comparisons in Sec. 4.3. Sec. 5 discusses the preprocessing and training challenges and the limitations that shaped our design, before Sec. 6 concludes the report with a summary of our findings.

2 Related Work

2.1 Novel View Synthesis

Novel View Synthesis (NVS) is the task of rendering a scene from unobserved viewpoints, given a set of posed input images, with applications in XR, robotic simulation, and digital twin construction. The classical pipeline utilizes Structure-from-Motion (SfM) to recover camera poses and a sparse point cloud [40], then densifies it with multi-view stereo [41], and renders novel views by reprojecting the textured reconstruction. While its geometry is fully explicit, photorealism is limited since reconstructions remain incomplete in weakly textured regions, and reprojected textures cannot reproduce view-dependent effects such as specular reflections [28].

Neural Radiance Fields (NeRF) [28] addresses these limitations by abandoning explicit geometry altogether, encoding the scene in a multilayer perceptron that maps position and viewing direction to color and density and compositing these along camera rays by volumetric rendering to produce photorealistic, view-consistent images. However, the volumetric integral requires hundreds of network evaluations per pixel, making training and rendering extremely slow compared to rasterized methods. In addition, the geometry is implicitly encoded in network weights, requiring network inference for every new frame of surface rendering.

3D Gaussian Splatting (3DGS) [22] addresses this trade-off between geometric explicitness and photorealism by representing the scene as anisotropic Gaussian kernels that are rasterized directly to the image, achieving real-time rendering with a quality competitive with NeRF. Its speed and fidelity made it the prevailing approach to real-time NVS. However, while individual kernels are explicit, the scene structure is not. Millions of independently optimized kernels carry no notion of the objects or surfaces that compose the scene. Since the photometric objective is ill-conditioned on flat, texture-less regions, which are prevalent in indoor environments, the optimization produces uneven, semi-transparent surfaces and floating artifacts detached from the true geometry [43, 19].

One line of work addresses this by constraining Gaussians to surfaces while preserving the Gaussian framework. 2D Gaussian Splatting (2DGS) [19] collapses each volumetric ellipsoid into an oriented planar disk that coincides with the local surface and admits exact ray-splat intersection, yielding view-consistent depth and normal maps alongside color. SuGaR [16] adds a regularizer that draws 3D Gaussians onto the surface and binds them to an extracted mesh. Two recent methods target the ill-conditioned indoor case directly: PlanarGS [20] obtains planar-region proposals from a vision-language segmentation model, fuses them across views, and adds planar and depth-normal supervision terms that flatten the Gaussians onto the detected surfaces, while 3D Gaussian Flats [43] changes the representation itself, modeling detected planar regions with plane-constrained 2D Gaussians and the rest of the scene volumetrically.

A second line replaces the Gaussian kernel itself, on the observation that the choice of rendering primitive determines which geometric structure the representation can express. MeshSplatting [17] optimizes an initially unconnected set of triangles, restores connectivity through a restricted Delaunay triangulation, and refines the result into a smooth,

opaque mesh that renders directly in real-time engines. Triangle Splatting [18] instead keeps the triangles deliberately disconnected, optimizing them as differentiable splats with softened boundaries and combining the adaptive density of Gaussian methods with surface primitives compatible with standard mesh pipelines. PlanarSplatting [44] adopts bounded planes as the primitive, splatting them into per-view depth and normal maps to reconstruct indoor surfaces with significantly less training time. 3D Superquadric Splatting [27] generalizes the Gaussian kernel to a superquadric rendered through its projected volumetric distance field, a strict super-set of Gaussian splats whose adjustable sharpness captures the edges, corners, and high-curvature geometry at which the smooth ellipsoid struggles.

Progress on both goals is measured predominantly on indoor benchmarks. ScanNet [7] provides more than 1500 RGB-D scans of indoor spaces, and ScanNet++ [57] extends it with high-resolution DSLR images, iPhone video streams, and laser-scanned ground-truth meshes, enabling the joint evaluation of photometric rendering quality and geometric reconstruction accuracy on which methods in this line are evaluated.

2.2 Geometric Primitives for Reconstruction

The methods of Sec. 2.1 treat primitives as rendering kernels whose purpose is image formation. A complementary body of work treats primitives as structural abstraction: a scene is described by a small set of parametric shapes, yielding a representation that is compact and interpretable, encoded by orders of magnitude fewer parameters than point clouds, meshes, or Gaussian sets, with each primitive an explicit geometric entity that can be named, edited, and reasoned about. Both properties carry practical weight: compactness enables deployment on memory-constrained hardware such as mobile AR/VR devices, and an explicit object-level geometric description directly supports downstream tasks that operate on geometry rather than appearance, such as robotic path planning and grasping [11], scene editing, and building-scale digital twins.

2.2.1 Volumetric Primitives

Solid objects are classically abstracted by volumetric primitives. Early work assembled them from cuboids [46], which are interpretable but too rigid to approximate curved geometry with few parts, motivating superquadrics: a continuous family that spans cuboids, cylinders, ellipsoids, and octahedra within a single eleven-parameter surface through two shape exponents and three scale parameters [1]. Superquadrics are defined by a closed-form inside–outside function, an algebraic expression of the shape parameters whose value is below one for points inside the surface, one on it, and above one outside. Because this function varies smoothly with all eleven parameters, the deviation of a point set from a candidate superquadric is a differentiable quantity, and fitting reduces to minimizing this residual by gradient descent. The same property enables learning-based prediction: Paschalidou et al. [32] train a network that regresses the parameters of a set of superquadrics under a reconstruction loss, such as the Chamfer distance between points sampled from the predicted primitives and the target shape, whose gradients propagate through the smooth parametrization into the network weights. Such models predict a decomposition in a single forward pass but encode only global shape and require category-

specific training data. Optimization-based methods such as EMS [26] instead fit each object individually from a point cloud through a probabilistic formulation, gaining accuracy and category independence but depending on clean and complete 3D input, which image-based reconstruction does not provide. Both paradigms operate on single objects, so applying them to a scene presupposes that its objects are already known. SuperDec [11] closes this gap by fitting superquadrics per object on top of a 3D instance segmentation, producing compact scene-level decompositions on ScanNet++ [57]. Throughout the family, the primitives encode shape without appearance and cannot synthesize images.

2.2.2 Planar Primitives

Indoor environments concentrate a large fraction of their visible surface in flat regions: walls, floors, and ceilings dominate the scene area [44]. A bounded plane describes such a region with only a normal, an offset, and a support extent, making planar primitives a substantially more parameter-efficient description of structural surfaces than general volumetric shapes. Learned detectors such as PlaneRCNN [25] predict planes and segmentation masks directly from a single image but require supervised plane annotations, which exist for only a few indoor datasets. PlanarSplatting [44] removes the learned detector, optimizing planes per scene through the differentiable splatting introduced in Sec. 2.1, supervised by monocular depth and normal cues, and reconstructs indoor surfaces within minutes without any 3D supervision. A related direction recovers parametric plane proxies from partial point clouds: PaCo [5] predicts planes together with their inlier points through sequence-to-sequence generation, enabling compact polygonal reconstruction under the occlusions and sensor gaps typical of real captures. Planes describe structural surfaces with very few parameters, but represent only flat geometry and, like the volumetric family, do not model appearance.

2.2.3 Hybrid Primitive Representations

The two families above encode geometry without appearance, while the Gaussian kernels of Sec. 2.1 provide the converse, photorealistic appearance without structure. A recent line of work therefore couples explicit geometric primitives with differentiable rendering to obtain an interpretable structural decomposition and photorealistic synthesis within a single representation. Differentiable Blocks World [29] optimizes a handful of textured superquadric meshes through rendering, demonstrating the principle but restricted to fewer than ten primitives and dependent on ground-plane alignment. PartGS [15], on which our pipeline builds, binds 2D Gaussians to the faces of superquadric meshes with shared pose parameters and optimizes both jointly through rendering, yielding an interpretable part decomposition alongside high-quality synthesis; as its authors note, it assumes background-free, object-centered inputs and represents all geometry with a single primitive family. 3D Gaussian Flats [43] brings the hybrid idea to full indoor scenes from the opposite direction, dynamically detecting planar regions during optimization and constraining their Gaussians to the fitted planes while representing non-planar content, such as furniture and clutter, with volumetric 3D Gaussians. Its planes act as constraints on the Gaussians rather than as parametric primitives of an object-level decomposition.

2.3 Monocular Depth Estimation

Recovering dense geometry from images has classically been the role of multi-view stereo, which densifies a Structure-from-Motion reconstruction by matching correspondences across views (Sec. 2.1). Correspondence matching, however, fails precisely in the weakly textured regions that make up most of an indoor scene, so the walls, floors, and ceilings our method targets are exactly where such reconstructions remain incomplete. Monocular depth estimation removes this constraint by inferring depth from a single image through priors learned from data, without any cross-view comparison, and therefore produces depth even in the texture-less regions where correspondence matching fails. It cannot, however, recover absolute metric scale, since scenes of different size can project to identical images, and the handling of this ambiguity has driven the development of the field.

Early deep methods predicted metric depth within a fixed domain such as indoor scenes or street views, trained on data from individual RGB-D or LiDAR sensors [10, 14, 2]. Eigen et al. [10] pioneered this formulation with a scale-invariant loss that separates scene layout from absolute magnitude. Because such models absorb the cameras, depth ranges, and scene statistics of their training sensor, their accuracy does not transfer to other domains.

Rather than resolving the scale ambiguity, the next generation made training invariant to it. MiDaS [37] showed that a scale-and-shift-invariant objective renders datasets with incompatible depth annotations jointly usable, and that mixing diverse sources in this way yields zero-shot transfer to unseen data. DPT [36] strengthened the same approach with a vision-transformer [9] backbone for globally coherent dense prediction, and Depth Anything scaled it to tens of millions of images through pseudo-labeling [55, 54]. The robustness gained this way comes at a price: the predicted depth is only relative, defined up to an unknown per-image scale and shift, which suffices for ordering pixels but not for lifting predictions into a common metric 3D space.

Restoring metric output without giving up this open-domain robustness became the goal of a subsequent line of work. For instance, ZoeDepth [3] builds on relative-depth pre-training and attaches domain-specific metric binning heads, Metric3D [60] instead maps training images into a canonical camera space that disentangles focal length from apparent depth, and UniDepth [33] estimates the camera jointly with the depth, removing the dependence on known intrinsics altogether. In parallel, the prediction target itself has been generalized from depth maps to full 3D geometry. Whereas earlier approaches recovered scene shape by decoupling the task into affine-invariant depth and separate camera parameter estimation [59], DUSt3R [51] demonstrated that per-pixel 3D point maps can be predicted end-to-end for image pairs, bypassing explicit camera recovery. MoGe [50] brought this representation to single images, predicting point maps in an affine-invariant form, a design its authors introduce specifically to eliminate the focal-distance ambiguity that hampers direct metric regression, and training them with an optimal alignment solver and multi-scale local supervision. MoGe-2 [49] completes this picture by restoring metric scale through a decoupled global scale branch that leaves the relative geometry untouched, sharpening detail through refined training labels, and predicting surface normals in the same forward pass.

2.4 Segmentation Model

Segmentation partitions visual data into meaningful regions. In *semantic segmentation*, every pixel or 3D point is assigned a category, but different objects of the same category are not distinguished. *Instance segmentation* additionally assigns a separate identity to every foreground countable object while ignoring the background, uncountable classes. *Panoptic segmentation* combines instance segmentation with semantic segmentation.

Early learning-based image instance segmentation was dominated by convolutional encoder–decoder architectures. U-Net [39] combines a contracting path that extracts increasingly contextual features with a symmetric decoding path and skip connections that recover precise spatial boundaries. With the advent of the Vision Transformer (ViT) [8], Transformer architectures were increasingly adopted for visual recognition and segmentation. Instead of relying exclusively on local convolutional operations, these models use self-attention to relate features across distant image regions and thereby incorporate global context into pixel-level predictions. SegFormer [52], for example, employs a hierarchical Transformer encoder to extract multi-scale features and a lightweight decoder to produce semantic predictions. Both families of models are usually trained for a fixed set of categories, limiting their ability to segment previously unseen object classes.

The Segment Anything Model (SAM) [23] introduced a different approach. Instead of predicting a fixed semantic label map, it produces class-agnostic object masks from geometric prompts such as points, boxes, or masks. SAM-2 [38] extends this promptable formulation from individual images to video by maintaining a memory of previous frames and propagating object masks through a sequence. SAM-3 [4] further introduces promptable concept segmentation, allowing short text phrases or visual examples to specify a concept and returning masks and persistent identities for all matching instances in images or videos. The ability to retain object identities across frames is particularly relevant to our use case, since reconstructing a consistent 3D scene requires observations of the same physical object to share an identity. However, when posed scene views are processed independently or differ substantially in viewpoint, additional cross-view consistency mechanisms may still be required.

Successive methods such as SAMPro3D [53] build upon this foundational model family, allowing placing prompts directly in a 3D point cloud and projecting each prompt into all posed image views in which it is visible. The corresponding SAM masks are therefore associated across views through their common 3D prompt, enabling training-free, class-agnostic 3D instance segmentation. While SAMPro3D [53] obtains 3D instances by projecting prompts into multiple 2D views, native 3D methods operate directly on point-cloud geometry.

Early point-cloud networks such as PointNet [34] process points directly while respecting their unordered structure, and PointNet++ [35] extends this formulation with hierarchical neighborhoods that capture local geometry. These architectures established strong backbones for 3D semantic segmentation, but instance segmentation additionally requires separating nearby objects, including objects belonging to the same semantic class. Building on such point-based feature representations, later 3D instance segmentation methods introduced explicit mechanisms for predicting object masks and assigning distinct instance

identities.

SPFormer [42] is a query-based 3D instance segmentation model that groups point-cloud features into geometrically coherent superpoints and decodes instance masks through superpoint-level cross-attention. OneFormer3D [24] extends the query-based formulation by jointly addressing semantic, instance, and panoptic point-cloud segmentation within a unified model. Despite their effectiveness, such query-based methods commonly rely on attention-based decoders, which can become computationally expensive for large point clouds, while the sparse and non-uniform distribution of 3D points also makes reliable query initialization difficult.

LaSSM [56], which we use in our pipeline, addresses these limitations through semantic-spatial query initialization and an efficient decoder based on local aggregation and state-space modeling. It groups nearby points into so called superpoints and initializes queries from those superpoints using both semantic evidence and spatial information, and then refines them with a coordinate-guided decoder that focuses on geometrically coherent regions. This makes LaSSM suitable for large indoor point clouds while producing both instance masks and their semantic categories. Applying it directly to our dense MoGe-2-TSDF reconstruction therefore provides the scene-level instance identities required for primitive initialization.

3 Method

3.1 Overview of the Pipeline

Figure 1 summarizes our method. Given posed RGB images with an SfM reconstruction, we first build a panoptic point cloud: MoGe-2 depth and normal predictions are scale-aligned to the SfM points, fused by TSDF integration into a dense surface $\mathcal{C}_M$, and segmented into 3D instances by LaSSM (Sec. 3.2). Each instance is then classified as volumetric or planar and seeded with superquadrics via SuperDec or with planar patches via RANSAC, yielding the hybrid representation $\mathcal{S} = \mathcal{P} \cup \mathcal{Q}$ whose surfaces carry 2D Gaussian skins $\mathcal{G}$ (Secs. 3.3 and 3.3.3). A geometric phase jointly optimizes the primitives and Gaussian colors under photometric and MoGe-2 depth–normal supervision; a photometric phase then freezes $\mathcal{S}$ and refines $\mathcal{G}$ as free 2DGS (Secs. 3.4.1 and 3.4.2). The method outputs an interpretable primitive decomposition of the scene and photorealistic novel-view renderings.

3.2 Point Cloud Initialization

3.2.1 Motivation and Requirements

The central goal of this section is to initialize the scene representation with appropriate primitives that correspond to meaningful parts of the real 3D scene. Instead of starting only from an unstructured point cloud, we want the initialization to already contain information necessary to distinguish foreground objects from the planar backgrounds. This is crucial because the primitive optimization directly optimizes its parameters to fit each segmented instance. A better initialization can guide superquadric and planar primitives

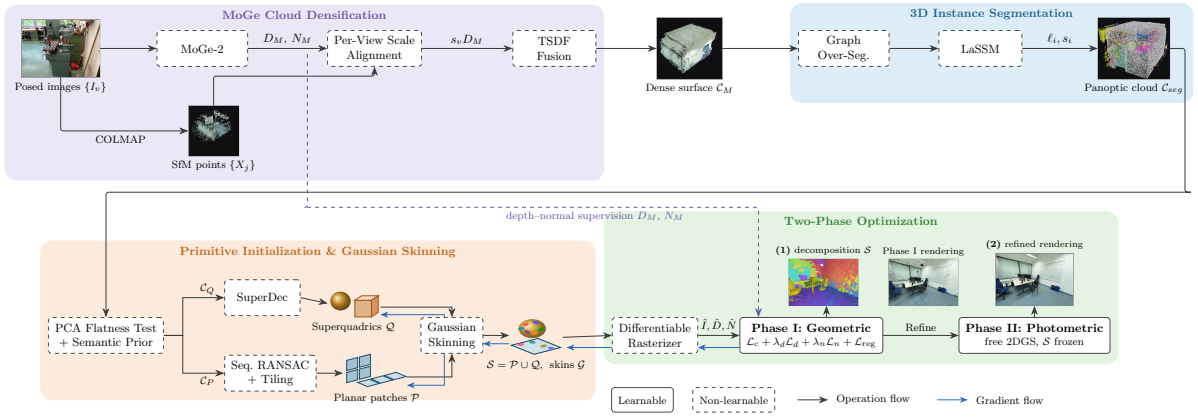


Figure 1: Overview of our pipeline. From posed RGB images, MoGe-2 depth and normal maps are scale-aligned to the SfM reconstruction and fused by TSDF integration into the dense surface $\mathcal{C}_M$, which LaSSM segments into the panoptic cloud $\mathcal{C}_{seg}$. A PCA flatness test with a semantic planar prior splits instances into volumetric points $\mathcal{C}_Q$ and planar points $\mathcal{C}_P$, seeding superquadrics $\mathcal{Q}$ via SuperDec and planar patches $\mathcal{P}$ via RANSAC. The hybrid scene $\mathcal{S} = \mathcal{P} \cup \mathcal{Q}$ is skinned with 2D Gaussians $\mathcal{G}$ and optimized in two phases through a differentiable rasterizer: a geometric phase fits the primitives jointly under photometric and MoGe-2 depth–normal supervision, and a photometric phase freezes $\mathcal{S}$ and refines $\mathcal{G}$ as free 2DGS. The output is (1) an interpretable primitive decomposition and (2) photorealistic novel-view renderings.

toward real-world entities such as chairs, tables, walls, floors, or cabinets while providing reasonable geometric interpretability of the whole scene.

Therefore, the preprocessing stage should produce a primitive-compatible decomposition of the reconstructed scene. In particular, it should satisfy the following requirements:

- It should produce a 3D point cloud with per-point instance labels.
- It should preserve the geometric information required to distinguish different primitive types.
- It should identify regions that correspond to individual scene foreground instances. In particular, points belonging to the same foreground object should be grouped together, while geometrically distinct objects should remain separated.
- It should identify background regions.

These requirements motivated our exploration of segmentation-based strategies for obtaining such a structured initialization, which are described in Appendix 6. We first lifted SAM3 [4] predictions from individual images into 3D and investigated both explicit graph-based cross-view association and video-based tracking to establish scene-level instance identities. However, the required complexity, threshold-sensitivity, and hard requirements to input data made those approaches not suitable for our problem. We next evaluated SAMPro3D [53], which establishes cross-view consistency by projecting 3D prompts into all visible views. However, it produced over-segmented results, required

scene-specific threshold tuning, and provided no semantic labels for distinguishing planes from superquadrics. We ultimately apply LaSSM [56] to segment instances directly in 3D on a dense MoGe-2–TSDF reconstruction [49, 50, 6, 30] (See Sec. 3.2.3).

3.2.2 MoGe Cloud Densification

To keep our method image-based, we reconstruct the point cloud directly from the scene images to guide our training. We found the Structure-from-Motion (SfM) point cloud too sparse to seed primitives or to support reliable instance segmentation (see Fig. 2). Thus, we densify the geometry by predicting dense per-view depth with the monocular geometry model MoGe-2 [49], built on a DINOv2 [31] ViT-L [9] backbone, and fuse the predictions into a single surface. For each training view v, MoGe-2 predicts in a single forward pass a per-pixel metric depth map D_M^v and camera-frame unit normal map N_M^v.

Two challenges arise that prevent per-view maps from achieving final single 3D cloud. First, monocular depth is metric only up to an unknown per-view scale i.e., different views do not share a common unit. Second, each view is predicted independently, so views that observe the same surface disagree about where it lies.

To resolve this scale ambiguity, we anchor each view to the COLMAP reconstruction with a single per-view scale factor s_v. Let $\{X_j\}$ be the COLMAP sparse points visible in view v, with metric camera-frame depths z_j and pixel projections $\pi(X_j)$. Matching these depths to MoGe-2's predictions at the same pixels yields

$$s_v \;=\; \mathrm{med}_j \, \frac{z_j}{D_M^v\big(\pi(X_j)\big)} \tag{1}$$

Rescaling $D_M^v \leftarrow s_v \, D_M^v$ then brings all views into a common metric scale.

With the views in a common scale, we lift each depth map into 3D world space, so each D_M^v becomes a partial observation of the scene surface. We then fuse them by truncated signed distance function (TSDF) integration [6, 30], resolving the second conflicting surface prediction issue. Each view contributes a signed-distance vote on a shared voxel grid (2 cm voxels, 6 cm truncation band, 8 m depth cutoff), and averaging the votes suppresses per-view noise. We extract the zero-crossing surface as a triangle mesh $\mathcal{M}_M = (\mathcal{C}_M, \mathcal{F}_M)$, whose vertices form the densified cloud $\mathcal{C}_M$ with per-vertex color and world-frame normals.

3.2.3 LaSSM 3D Instance Segmentation

Our previous experimental methods in segmenting and lifting 2D images to 3D point cloud (please refer to Appendix for more details) result in fragmented instances that are not suitable for primitive initialization (see Sec. 3.3.3), or use hand-tuned thresholds that do not generalize. We therefore segment point clouds into instances directly.

To find a strong 3D instance segmentation model, we consulted the ScanNet++ benchmark leaderboard.[1] The three top-ranked entries are unusable in practice: Volt-SPFormer (mean AP 0.360, Volt backbone [58] with the SPFormer head [42]) releases no weights for

[1] `https://scannetpp.mlsg.cit.tum.de/scannetpp/benchmark/insseg`

(a) COLMAP SfM cloud (b) MoGe–TSDF surface $\mathcal{C}_M$

Figure 2: Cloud densification for ScanNet++ scene `39f36da05b`. **(a)** The COLMAP SfM cloud is a noisy scatter of keypoints with floaters and gaps and lacking a coherent surface. **(b)** TSDF fusion of the per-view MoGe-2 depth maps yields the dense, denoised, connected surface $\mathcal{C}_M$ on which primitive initialization and instance segmentation can operate. Both are rendered at 120k points.

the full split, and ACGP and PointComp provide no public code. We thus use **LaSSM** [56] (mean AP 0.324), the highest-ranked method with public code and open weights.

TSDF fusion of the MoGe-2 depth maps D_M (Sec. 3.2.2) yields, with the cloud $\mathcal{C}_M$, a triangle mesh $\mathcal{M}_M = (\mathcal{C}_M, \mathcal{F}_M)$, each vertex x_i carrying a surface normal n_i derived from the MoGe-2 normals N_M. We over-segment $\mathcal{M}_M$ into S superpoints by graph-based segmentation [12], $\pi : \{1, \dots, |\mathcal{C}_M|\} \to \{1, \dots, S\}$, and apply LaSSM,

$$\Phi : \ (\mathcal{M}_M, \pi) \ \longmapsto \ \big\{(\ell_i, s_i)\big\}_{i=1}^{|\mathcal{C}_M|}, \tag{2}$$

assigning each $x_i \in \mathcal{C}_M$ an instance label $\ell_i \in \{1, \dots, K\} \cup \{-1\}$ and a semantic label $s_i \in \{1, \dots, C\} \cup \{-1\}$, where -1 marks background. The result is the segmented and labelled cloud

$$\mathcal{C}_{seg} = \big\{(x_i, n_i, \ell_i, s_i)\big\}_{i=1}^{|\mathcal{C}_M|}, \tag{3}$$

whose instance labels are used to initialize the per-object primitives and whose semantic labels can be useful for planar-volumetric classification.

Please note that LaSSM is trained on laser-scan meshes but applied here to the noisier MoGe-2/TSDF reconstruction. Despite the domain gap, we found that the instance structure suffices to separate compact objects from planar surfaces for the primitive initialization for our method to converge. Figure 3 shows a representative result.

3.3 Scene Representation

3.3.1 Geometric Representation

To break down scenes in a way that captures the geometric differences between structural elements (e.g., walls, floors, and ceilings) and foreground objects (e.g., furniture or smaller items), we propose a novel dual-primitive representation.

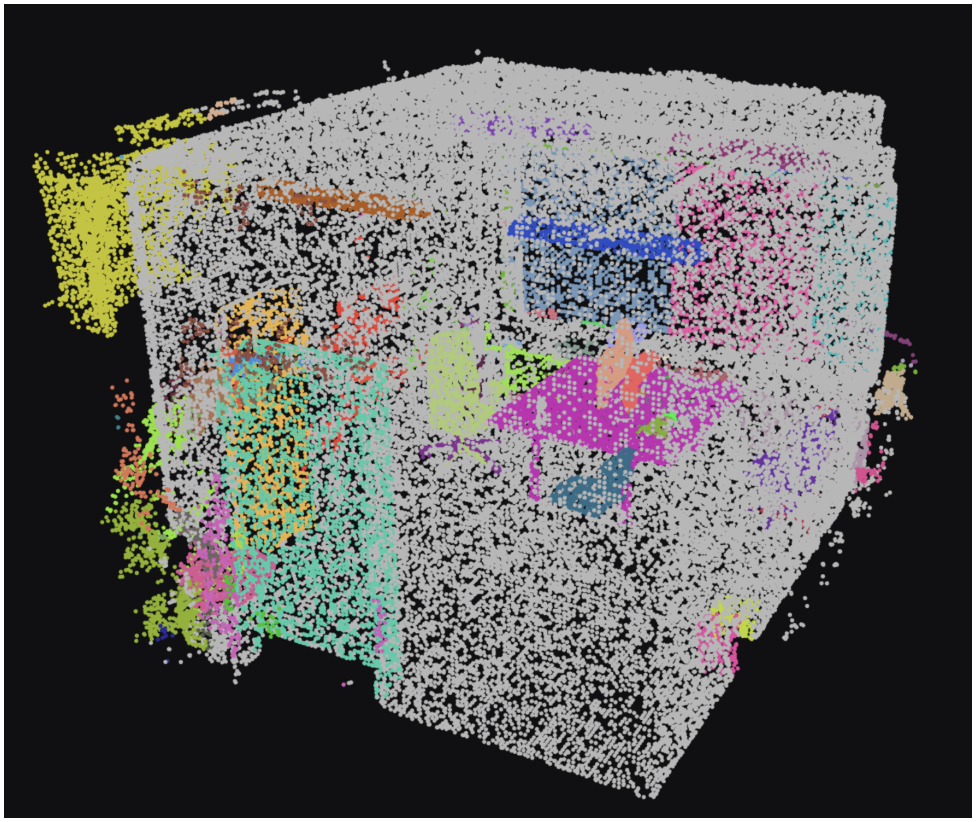


Figure 3: LaSSM instance segmentation of the MoGe–TSDF reconstruction of scene `39f36da05b`, colored by predicted instance ℓ_i, gray denotes background instances ($\ell_i = -1$). Despite the out-of-distribution input, instances separate compact objects (superquadrics) from planar structures (walls and floors).

Definition 3.1. Let $\mathcal{P} = \{P_i\}_{i=1}^{N_p}$ and $\mathcal{Q} = \{Q_i\}_{i=1}^{N_q}$ be sets of planar and superquadric primitives respectively and $\mathcal{S} = \mathcal{P} \cup \mathcal{Q}$ be our geometric scene representation. We parametrize each element as

$$Q_i = (\varepsilon_i, r_i, q_i, p_i, \alpha_i), \quad P_i = (r_i, q_i, p_i, \alpha_i), \tag{4}$$

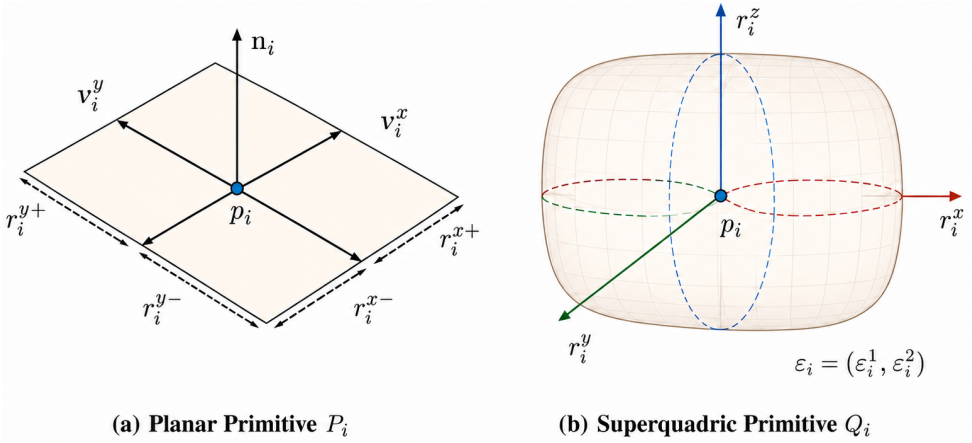
with center $p_i \in \mathbb{R}^3$, unit quaternion $q_i \in \mathbb{R}^4$, and opacity $\alpha_i \in [0, 1]$. We express a world point $x \in \mathbb{R}^3$ in the local frame of a primitive as $(x', y', z')^\top = R(q_i)^\top (x - p_i)$, where $R(q_i)$ is the rotation matrix associated with q_i.

For planar primitives, we follow the double-radius parametrization of PlanarSplatting [44], $r_i = (r_i^{x+}, r_i^{x-}, r_i^{y+}, r_i^{y-}) \in \mathbb{R}_+^4$, which bounds the extent of P_i independently along the four in-plane directions. In the local frame, P_i is the axis-aligned rectangle in the $z' = 0$ plane:

$$z' = 0, \quad -r_i^{x-} \le x' \le r_i^{x+}, \quad -r_i^{y-} \le y' \le r_i^{y+}. \tag{5}$$

For superquadric primitives, we follow SuperDec [11], where the radii $r_i = (r_i^x, r_i^y, r_i^z) \in \mathbb{R}_+^3$ and shape exponents $\varepsilon_i = (\varepsilon_i^1, \varepsilon_i^2)$ define an implicit surface. In the local frame, Q_i satisfies:

$$\left(\left(\frac{x'}{r_i^x} \right)^{2/\varepsilon_i^2} + \left(\frac{y'}{r_i^y} \right)^{2/\varepsilon_i^2} \right)^{\varepsilon_i^2/\varepsilon_i^1} + \left(\frac{z'}{r_i^z} \right)^{2/\varepsilon_i^1} = 1. \tag{6}$$



(a) Planar Primitive P_i **(b) Superquadric Primitive Q_i**

Figure 4: Geometric parametrization of the two primitive types. The planar primitive P_i is represented by its center p_i, orientation, and four independent in-plane radii $(r_i^{x+}, r_i^{x-}, r_i^{y+}, r_i^{y-})$. The superquadric primitive Q_i is represented by its center, orientation, axis radii (r_i^x, r_i^y, r_i^z), and shape exponents $\varepsilon_i = (\varepsilon_i^1, \varepsilon_i^2)$.

Figure 4 illustrates the parametrization of the planar and superquadric primitives used in our scene representation.

During optimization, those parameters would be updated to fit the scene geometry, leveraging both photometric and geometric cues.

Definition 3.2. For every primitive $P_k \in \mathcal{S}$ we define a signed distance $d_k(\mathbf{x})$, which is negative inside, positive outside and zero on its surface, measuring the distance from a point $\mathbf{x}$ to P_k. Since a superquadric has no closed-form Euclidean SDF, we derive the radial distance [47] $d_k(\mathbf{x}) = q_k(\mathbf{x})^{\varepsilon_1/2} - 1$ from its inside-outside function $q_k(\mathbf{x})$, where $q_k(\mathbf{x})$ is the inside-outside function from the Left Hand Side of Equation 6; for a plane, d_k is the analytic SDF of a thin rectangular slab.

3.3.2 2D Gaussian skins for Appearance Modeling

Inspired by GaMeS [48] and PartGS [15], we decouple appearance from geometry via *Gaussian Skinning*. This process consists on sampling a set of 2D Gaussian splats on the surface of each primitive, which is then optimized to reproduce the photometric appearance of the scene.

Definition 3.3. We define a set of 2D Gaussians

$$\mathcal{G} = \{g_j\}_{j=1}^{N_g}, \quad \text{where } g_j = (r_j, p_j, s_j, q_j, c_j), \tag{7}$$

with splat radius $r_j \in \mathbb{R}_+$, center $p_j \in \mathbb{R}^3$, tangential scales $s_j \in \mathbb{R}^2$, orientation $q_j \in \mathbb{R}^4$, and view-dependent color c_j represented with spherical harmonics.

Each Gaussian is associated with a primitive from $\mathcal{S}$ and lays in a triangular face T. For planes, those faces are just obtained via a trivial triangular subdivision, while for superquadrics we use an icosphere to approximate their curved surface. This triangular face provides a surface to anchor the splat, allowing for p_j in a compact way via barycentric

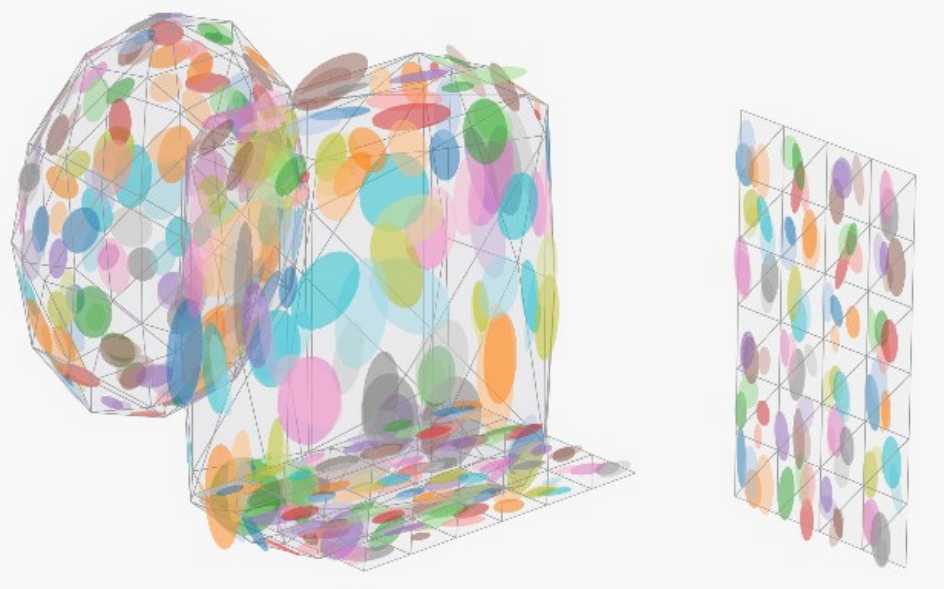


Figure 5: Illustration of Gaussian Skinning for the two primitive types. A set of 2D Gaussian splats is anchored to the triangulated surface of each primitive. Planar primitives are subdivided directly into triangular faces, whereas superquadric surfaces are approximated by an icosphere triangulation.

coordinates, as well as a local frame to define its orientation, and an opacity α_i , which is shared across all splats of that primitive and its required for the alpha compositing of Gaussian splatting. These attributes are initialized randomly for each of the N_T Gaussians sampled in each face.

Figure 5 illustrates the Gaussian Skinning process for planar and superquadric primitives.

Note that the gradients experimented by the Gaussian splats, based solely in photometric losses, are backpropagated to the vertices of the triangularization and back to the primitive parameters, allowing for a joint optimization of both geometry and appearance. For mathematical details of loss backpropagation, we recommend the reader to refer to [15].

3.3.3 Primitive Initialization

Both primitive families $\mathcal{P}$ and $\mathcal{Q}$ are initialized from the segmented point cloud $\mathcal{C}_{seg}$ produced in preprocessing (Sec. 3.2.3). Each instance is first classified as *volumetric* or *planar* by a PCA [21] flatness test. For an instance with points $\{x_i\}_{i=1}^{n}$ and centroid $\bar{x}$, we form the covariance $\Sigma = \frac{1}{n}\sum_i (x_i - \bar{x})(x_i - \bar{x})^\top$ with eigenvalues $\lambda_1 \leq \lambda_2 \leq \lambda_3$ and mark the instance planar if

$$\frac{\lambda_1}{\lambda_1 + \lambda_2 + \lambda_3} < \tau, \qquad \tau = 0.02, \tag{8}$$

i.e. when the point set has negligible extent along its shortest principal axis. A semantic override additionally marks instances of inherently planar object classes (door, window, whiteboard, picture, etc.) as planar. After this process, the remaining superquadric

instances and the rest of the cloud form a partition $\mathcal{C}_Q \cup \mathcal{C}_P = \mathcal{C}_{seg}$, $\mathcal{C}_Q \cap \mathcal{C}_P = \emptyset$, from which superquadric and planar instances are subsequently seeded.

Superquadric Initialization To get a rich initialization from each volumetric instance, we leverage SuperDec [11] as a strong prior that maps an instance point cloud $c_q \subset \mathcal{C}_Q$ to a small set of superquadrics with an existence probability. We keep the primitives whose existence exceeds a threshold and set a limit of eight superquadrics per instance. This yields a compact yet expressive initialization that already captures parts and concavities. When the instance is too small for SuperDec, we fall back to a trivial single-superquadric fit where we place an ellipsoid on the centroid of the instance.

Planar Initialization Large flat structures are represented not by one primitive but by many small planar patches. Following PlanarSplatting [44], we extract dominant planes from $\mathcal{C}_P$ with sequential RANSAC [13] and then tile each plane's inliers into a regular grid, with each tile becoming one single plane primitive. This local patchwork adapts to irregular room boundaries better than a handful of global planes.

3.4 Optimization framework

3.4.1 Geometric phase

In the first phase of the optimization, we jointly optimize the parameters of the primitives in $\mathcal{S}$ and the colors of the Gaussians in $\mathcal{G}$, while the remaining Gaussian parameters stay frozen. The objective combines photometric consistency with explicit geometric supervision,

$$\mathcal{L} \;=\; \mathcal{L}_c \;+\; \lambda_d \mathcal{L}_d \;+\; \lambda_n \mathcal{L}_n + \lambda_{\text{pars}} \mathcal{L}_{\text{pars}} + \lambda_{\text{attach}} \mathcal{L}_{\text{attach}} + \lambda_{\text{sep}} \mathcal{L}_{\text{sep}}. \tag{9}$$

Photometric supervision. $\mathcal{L}_c$ combines an $\mathcal{L}_1$ term on the rendered RGB values with the D-SSIM term of 3DGS [22], with the standard weighting $\mathcal{L}_c = (1-\lambda)\mathcal{L}_1 + \lambda\,\mathcal{L}_{\text{D-SSIM}}$, $\lambda = 0.2$.

Geometric supervision. The terms $\mathcal{L}_d$ and $\mathcal{L}_n$ align rendered depth and normal maps against the MoGe-2 [49] maps D_M and N_M. Since MoGe-2 predicts normals in the camera frame, we rotate them into world coordinates, $\mathbf{n}_M(\mathbf{p}) = R\,\mathbf{n}_M^{\text{cam}}(\mathbf{p})$, with R the camera-to-world rotation of the training view. On the rendering side, front-to-back α-blending of the skinned 2D Gaussians yields the expected depth and blended normal

$$\hat{D}(\mathbf{p}) \;=\; \frac{\sum_i w_i\, z_i}{\sum_i w_i}, \qquad \hat{\mathbf{n}}(\mathbf{p}) \;=\; \frac{\sum_i w_i\, \mathbf{n}_i}{\left\|\sum_i w_i\, \mathbf{n}_i\right\|}, \tag{10}$$

where z_i and $\mathbf{n}_i$ are the depth and world-space normal of the i-th ray-splat intersection at pixel $\mathbf{p}$, and $w_i = \alpha_i \prod_{j<i}(1-\alpha_j)$ are the blending weights. Both terms are evaluated on the valid-pixel set $\mathcal{V} = \{A(\mathbf{p}) > 0.5\}$, where $A(\mathbf{p}) = \sum_i w_i$ is the accumulated opacity.

The depth term is a median-normalized, scale-invariant $\mathcal{L}_1$,

$$\mathcal{L}_d \;=\; \frac{1}{|\mathcal{V}|} \sum_{\mathbf{p}\in\mathcal{V}} \left| \frac{\hat{D}(\mathbf{p})}{\text{med}_{\mathcal{V}}(\hat{D})} \;-\; \frac{D_M(\mathbf{p})}{\text{med}_{\mathcal{V}}(D_M)} \right|, \tag{11}$$

where $\mathrm{med}_{\mathcal{V}}(\cdot)$ is the median over $\mathcal{V}$, with the rendered median detached during back-propagation. This makes the term invariant to the global scale ambiguity of monocular depth.

The normal term combines an $\mathcal{L}_1$ and a cosine alignment penalty,

$$\mathcal{L}_n \;=\; \frac{1}{|\mathcal{V}|}\sum_{\mathbf{p}\in\mathcal{V}} \left[\tfrac{1}{3}\left\|\hat{\mathbf{n}}(\mathbf{p}) - \mathbf{n}_M(\mathbf{p})\right\|_1 \;+\; \left(1 - \hat{\mathbf{n}}(\mathbf{p})\cdot\mathbf{n}_M(\mathbf{p})\right)\right]. \tag{12}$$

We use $\lambda_d = 1.5$ and $\lambda_n = 4$ in our experiments.

Primitive regularization. In some scenarios, the optimization can converge to configurations where (I) too many primitives cover a single surface in small patches, (II) primitives drift off the geometry they should fit, or (III) several primitives overlap heavily on a single surface. To alleviate (I), we introduce the following weighted parsimony term, applied to each family $\mathcal{F} \in \{\mathcal{Q}, \mathcal{P}\}$,

$$\mathcal{L}^{\mathcal{F}}_{\text{pars}} \;=\; \frac{1}{|\mathcal{F}|}\sum_{k\in\mathcal{F}} (1 - \exp(-\delta_k/\rho))\,\sqrt{\alpha_k}, \tag{13}$$

where $\delta_k = \frac{1}{\kappa}\sum_{\mathbf{x}\in\mathcal{N}_\kappa(k)}\left[d_k(\mathbf{x})\right]_+$ is the detached distance from P_k to its $\kappa = 16$ nearest observed points $\mathcal{N}_\kappa(k) \subset \mathcal{C}_{\mathcal{F}}$ and $\rho = 0.1$. Barely contributing primitives with low opacity are thus rapidly pushed to being fully transparent and ultimately culled when their opacity falls under a threshold. The weight term $1 - \exp(-\delta_k/\rho)$ spares primitives that still rest on the point cloud and applies the pressure only to floaters.

For (II) and (III), we adapt ideas from SuperFlex [45] to our scene-level, noisy setting. To keep each primitive anchored to the geometry it should explain (II), every point labeled with a family, $\mathbf{x} \in \mathcal{C}_{\mathcal{F}}$, is drawn onto its nearest surface,

$$\mathcal{L}^{\mathcal{F}}_{\text{attach}} = \frac{1}{|\mathcal{C}_{\mathcal{F}}|}\sum_{\mathbf{x}\in\mathcal{C}_{\mathcal{F}}} \left[\,\psi_\tau\big(\min_{k\in\mathcal{F}} d_k(\mathbf{x})\big)\,\right]_+, \tag{14}$$

where $[\,\cdot\,]_+ = \max(\cdot, 0)$ and $\psi_\tau(u) = \tau\tanh(u/\tau)$ is a truncated activation with $\tau = 0.1$, reducing the effect of noisy MoGe points. To discourage primitives from growing into one another (III), we sample points $\mathcal{X}_k$ inside each source primitive $k \in \mathcal{F}$ and penalize intrusion into the nearest other primitive, taken over all of $\mathcal{S}$,

$$\mathcal{L}^{\mathcal{F}}_{\text{sep}} = \frac{1}{\sum_{k\in\mathcal{F}}|\mathcal{X}_k|}\sum_{k\in\mathcal{F}}\sum_{\mathbf{x}\in\mathcal{X}_k} \sigma\big(-d_k(\mathbf{x})/\gamma\big)\,\alpha_k\,\psi_\tau\big(\big[-\min_{j\in\mathcal{S},\,j\neq k} d_j(\mathbf{x})\,\big]_+\big), \tag{15}$$

where σ is the sigmoid, γ a temperature and α_k the source opacity; the indicator $\sigma(-d_k/\gamma)$ limits the penalty to points inside their source primitive, and the smooth penetration term fires at the onset of overlap. Since the nearest other primitive is searched over $\mathcal{S}$, the superquadric term also keeps superquadrics off walls and floors. We use We use $\lambda_{\text{pars}} = 0.02$, $\lambda_{\text{attach}} = 20$, $\lambda_{\text{sep}} = 5$ and $\gamma = 5 \times 10^{-3}$.

3.4.2 Photometric phase

Once the geometric phase has converged to a primitive decomposition $\mathcal{S}$, we detach the Gaussians in $\mathcal{G}$ from the primitive surface faces and optimize them freely under the standard 2DGS framework [19]. The primitives are frozen during these phase, with each Gaussian retaining the identity of its parent primitive, which is propagated through densification so that the part decomposition is preserved while photometric quality is refined. The optimization itself is unmodified 2DGS: photometric $\mathcal{L}_1$ + D-SSIM supervision with its depth-distortion and normal-consistency regularizers, and adaptive density control to recover fine detail the coarse primitive geometry cannot represent.

4 Experiments

We validate our method on large-scale indoor scene reconstruction. Sec. 4.1 describes the datasets, baselines, training and inference settings, and evaluation metrics. Sec. 4.2.1 reports geometry reconstruction quality against the official aligned mesh. Sec. 4.2.2 evaluates rendering quality on held-out test views. Sec. 4.2.3 analyzes representation compactness in terms of primitive count. Sec. 4.3 provides visual comparisons on representative scenes.

4.1 Experimental Setup

4.1.1 Datasets

We chose **ScanNet++** [57] as our dataset for evaluation, using the **iPhone** sensor stream with undistorted video frames, COLMAP [40] poses, and aligned ground-truth meshes. ScanNet++ is a large-scale indoor-scene benchmark that extends ScanNet [7] with higher-quality capture and richer annotations. Each scene is a full apartment-scale room recorded with multiple sensor modalities, including high-resolution DSLR photographs, iPhone video streams, and a dense laser reconstruction.

In this work we adopt the **iPhone subset**, which provides a continuous sequence of undistorted frames together with COLMAP camera poses under a NeRF-style pinhole camera model. For every scene, the dataset additionally releases an official *aligned ground-truth mesh*, obtained by registering a high-resolution laser scan to the capture coordinate frame at 5 cm voxel resolution.

ScanNet++ scenes are structurally heterogeneous: large planar regions (walls, floors, ceilings, cabinets) coexist with cluttered non-planar objects (chairs, tables, appliances) in a shared coordinate frame. This mixture is well matched to our method, which assigns explicit planes to architectural surfaces and deformable superquadrics to compact objects, rather than forcing the entire scene into a single primitive family.

4.1.2 Baselines

To thoroughly validate our method, we compare against a set of representative state-of-the-art approaches, organized according to the primary evaluation aspect they target. For geometry reconstruction, we select two planar-primitive baselines: **PlanarSplatting** [44] and **3D Gaussian Flats** [43], as they are specifically designed to produce explicit surface geometry and have demonstrated strong performance on indoor scenes. For novel view synthesis, we choose three unstructured rendering baselines: **3D Gaussian Splatting** [22], **2D Gaussian Splatting** [19], and **MeshSplatting** [17], which represent the current leading paradigms for high-fidelity appearance modeling. Below we briefly describe each method, grouped by its evaluation role.

Geometry Baselines:

- **PlanarSplatting [44]:** A splatting framework that represents 3D scenes through a set of bounded planar primitives. It leverages foundation models to obtain depth

and normal maps for supervision. Through a differentiable plane rasterizer, it is able to efficiently reconstruct coarse geometry, obtaining accurate results on ScanNet++ in less than three minutes of training time per scene.

- **3D Gaussian Flats [43]:** A hybrid 2D/3D Gaussian representation that jointly optimizes constrained planar Gaussians for flat surfaces and freeform Gaussians for the rest. It is dynamic in refining plane regions and generating high-quality mesh extractions, thereby providing an excellent benchmark that utilizes planes but lacks superquadric primitives.

Novel View Synthesis Baselines:

- **2D Gaussian Splatting [19]:** Surfel representation that reduces 3D volumes to oriented 2D Gaussian disks. It uses consistent geometry from view through ray-splat intersections and normal consistency for reconstructing the surfaces and retaining good quality in rendering.
- **3D Gaussian Splatting [22]:** The standard ellipsoidal splatting approach that achieves real-time photorealistic rendering through differentiable rasterization of 3D Gaussians.
- **MeshSplatting [17]:** A differentiable mesh-based renderer that jointly optimizes geometry and appearance. It enforces connectivity via restricted Delaunay triangulation, producing watertight meshes that render efficiently in real-time engines while achieving strong NVS performance.

This dual-track comparison allows us to separately assess the geometric fidelity and the rendering quality of our hybrid representation against the most competitive methods in each domain.

4.1.3 Training and Inference Configuration

We follow the evaluation protocol of **3D Gaussian Flats** [43] for scene partitioning and resolution settings. Specifically, we use a fixed set of 11 scenes held out from all training runs and the same scene list is used for every compared method. The IDs of the 11 scenes are 0a7cc12c0e, 0cf2e9402d, 0e75f3c4d9, 1ae9e5d2a6, 1b75758486, 1c4b893630, 2e74812d00, 4c5c60fa76, 4ea827f5a1, 5748ce6f01, 7079b59642. Within each scene, frames are sorted by filename and partitioned, reserving every 8^{th} frame for testing and the remainder for training. All images are used at $2\times$ downsampled resolution relative to the undistorted iPhone export, consistent with published baseline configurations. All experiments are conducted on a single NVIDIA RTX5090 GPU with 32 GB memory.

We train both phases to 15000 iterations, with a learning rate of $2.5 \cdot 10^{-3}$ on the SH coefficients of the gaussians on both phases. On the geometric phase, we use a learning rate of $5 \cdot 10^{-3}$ on the geometry and 10^{-3} on the opacity, while on the photometric phase we use the 2DGS defaults of $5 \cdot 10^{-2}$ for opacity, $5 \cdot 10{-}3$ for scaling and 10^{-3} for the rotation of the gaussians. We use PlanarSplatting planar merging for obtaining our final mesh after geometric phase optimization, and concatenate the superquadrics from $\mathcal{Q}$ to it.

4.1.4 Evaluation Metrics

We evaluate our method along three dimensions: geometric reconstruction accuracy, novel view synthesis quality, and representation compactness.

Geometry Reconstruction. For each scene, we uniformly sample $N = 300{,}000$ points from both the predicted mesh surface $\hat{\mathcal{M}}$ and the aligned ground-truth mesh $\mathcal{M}^\star$, yielding point sets $\hat{\mathcal{P}}$ and $\mathcal{P}^\star$, respectively.

Chamfer Distance (↓): Following PlanarSplatting [44] and 3D Gaussian Flats [43], we report the L1 Chamfer mean:

$$\mathrm{CD} = \frac{1}{2}\left(\mathbb{E}_{\mathbf{p}\in\hat{\mathcal{P}}}\, d(\mathbf{p}\to\mathcal{P}^\star) + \mathbb{E}_{\mathbf{q}\in\mathcal{P}^\star}\, d(\mathbf{q}\to\hat{\mathcal{P}})\right), \tag{16}$$

which measures the bidirectional surface discrepancy between the reconstruction and the ground truth.

Completeness (↓): We report the one-sided distance from ground truth to prediction:

$$\text{Completeness} = \mathbb{E}_{\mathbf{q}\in\mathcal{P}^\star}\, d(\mathbf{q}\to\hat{\mathcal{P}}), \tag{17}$$

which quantifies how much of the ground-truth surface is covered by the reconstruction. High completeness errors reveal *missing* geometry, e.g. uncovered walls, floors, or object parts. These things are particularly informative in room-scale scenes where large planar structures dominate the surface area.

Accuracy (↓): We report the one-sided distance from prediction to ground truth:

$$\text{Accuracy} = \mathbb{E}_{\mathbf{p}\in\hat{\mathcal{P}}}\, d(\mathbf{p}\to\mathcal{P}^\star), \tag{18}$$

which quantifies how close reconstructed surface points lie to the ground truth. Low accuracy indicates geometric *overshoot*: floaters, thickened surfaces, or geometry extending beyond the true shape. Together with completeness (Eq. (17)), accuracy decomposes the Chamfer distance into two interpretable one-sided terms.

F-score (↑): At a distance threshold of $\tau = 0.05\,\mathrm{m}$, we define precision as the proportion of predicted samples within τ of the ground truth, and recall as the proportion of ground-truth samples within τ of the prediction. The F-score is computed as their harmonic mean.

Novel View Synthesis. For the held-out test frames, we report PSNR (↑), SSIM (↑), and LPIPS (↓) averaged over all test views and scenes. All methods are evaluated on the identical set of test frames within each scene to ensure a fair comparison.

Compactness. To assess the representation efficiency, we consider along two complementary axes. **Rendering compactness** counts the primitives that participate in the final differentiable rasterizer: Gaussian surfels (#G) for 3D Gaussian Splatting [22], 2D Gaussian Splatting [19], and Primimix (our method), and triangles (#T) for MeshSplatting [17]. Because these primitive types differ, we do not merge them into a single scalar.

Instead, we report each method with its native rendering primitive, together with on-disk storage. **Structural compactness** applies to Primimix only: we additionally report the number of explicit plane and superquadric blocks (#Plane, #SQ) of our method, which quantify the interpretable geometric decomposition not available in Gaussian-only baselines.

4.2 Quantitative Results

4.2.1 Geometry Reconstruction Performance

We evaluate mesh reconstruction on the 11 scenes from the ScanNet++ [57] iPhone subset. Predicted meshes are compared against `mesh_aligned_0.05.ply` using $N = 300{,}000$ surface samples per mesh.

Table 1 reports L1 Chamfer distance, accuracy, and completeness over all test scenes. Lower is better for all three distance-based metrics. Additionally, we report F-score at a 5 cm threshold, where higher is better. Accuracy and completeness together provide a finer-grained diagnosis than Chamfer distance alone: a method with low Chamfer Distance may still suffer from missing surfaces (high completeness) or surface overshoot (high accuracy).

Table 1: Geometry reconstruction results on the ScanNet++ [57] dataset. All metrics are computed from 300,000 uniformly sampled points on each mesh surface. Chamfer distance, accuracy, and completeness are reported in centimeters; F-score is evaluated at a 5 cm threshold.

Method	Chamfer Distance ↓	Accuracy ↓	Completeness ↓	F@5 ↑
PlanarSplatting [44]	<u>10.99</u>	9.58	<u>12.41</u>	44.70
3D Gaussian Flats [43]	18.60	**4.60**	32.59	<u>50.24</u>
Primimix (Ours)	**6.63**	<u>7.00</u>	**6.26**	**62.80**

According to Table 1, PlanarSplatting [44] obtains the best baseline Chamfer distance (10.99 cm) and the highest accuracy (9.58 cm). However, the completeness error of this model (12.41 cm) implies that some missing geometry exists due to modeling using only planes for compact objects. 3D Gaussian Flats [43] shows the opposite failure mode: low accuracy (4.60 cm) but very high completeness error (32.59 cm), indicating large under-reconstructed regions despite locally accurate predictions. Primimix outperforms both baselines on overall geometric fidelity and significantly reduces missing structure in comparison to 3D Gaussian Flats, improving Chamfer distance to 6.63 cm and completeness to 6.26 cm and F-score to 62.80 while maintaining accuracy at 7.00 cm. These results support our core claim that combining planes and superquadrics is more effective for room-scale reconstruction than either primitive type alone.

4.2.2 Novel View Synthesis Performance

For novel view synthesis, we evaluate all methods under an identical data and initialization protocol on the 11 ScanNet++ [57] iPhone scenes. **3D Gaussian Splatting** [22], **2D**

Gaussian Splatting [19], and **MeshSplatting** [17] are trained on the same undistorted iPhone frames, the same `llffhold=8` train/test partition, and the same 2× downsampled resolution. All three unstructured baselines are initialized from the **COLMAP [40] sparse point cloud** bundled with each scene, ensuring that differences in rendering quality reflect the representation and optimization strategy rather than inconsistent camera calibration or point-cloud seeding. Each baseline is trained for 30,000 iterations and evaluated on the held-out test frames only. MeshSplatting additionally enforces mesh connectivity through restricted Delaunay triangulation, while 3D Gaussian Splatting and 2D Gaussian Splatting optimize free-form Gaussian primitives without explicit surface structure. Our method is evaluated under the same test views; it produces the Gaussian surfels used for final rendering.

Table 2 summarizes novel view synthesis quality on the held-out iPhone test frames for all 11 evaluation scenes.

Table 2: Novel view synthesis results on the ScanNet++ [57] dataset. We report PSNR, SSIM, and LPIPS averaged over all held-out test frames across the 11 evaluation scenes.

Method	PSNR ↑	SSIM ↑	LPIPS ↓
2D Gaussian Splatting [19]	<u>19.22</u>	<u>0.8445</u>	<u>0.2663</u>
3D Gaussian Splatting [22]	**19.62**	**0.8627**	**0.2269**
MeshSplatting [17]	18.70	0.7845	0.3776
Primimix (Ours)	18.74	0.8206	0.2936

From Table 2, 3D Gaussian Splatting [22] achieves the best rendering scores (19.62 dB, 0.8627, 0.2269), followed by 2D Gaussian Splatting [19]. MeshSplatting [17] attains the lowest PSNR (18.70 dB) and the highest LPIPS (0.3776). Primimix reaches 18.74 dB PSNR, 0.8206 SSIM, and 0.2936 LPIPS, trailing the two Gaussian splatting baselines but clearly outperforming MeshSplatting on all three metrics, especially LPIPS. The gap to 3D Gaussian Splatting (∼0.9 dB PSNR) indicates that fully unstructured optimization still has an advantage in fitting view-dependent appearance, whereas our Stage II Gaussian-skin refinement recovers much of the perceptual quality despite the underlying scene being constrained by explicit plane and superquadric blocks. Overall, Primimix does not yet match the best unstructured splatting methods in novel view synthesis, but it remains competitive with mesh-based rendering while offering structured geometry (Sec. 4.2.1) and an interpretable decomposition (Table 4).

4.2.3 Compactness Analysis

Compactness is a central motivation of our hybrid representation: explicit planes and superquadrics should explain a scene with far fewer degrees of freedom than unstructured Gaussian or triangle soup. We compare against the novel view synthesis baselines from Sec. 4.2.2. For **3D Gaussian Splatting** [22] and **2D Gaussian Splatting** [19], we count Gaussian surfels in the final `point_cloud.ply`. For **MeshSplatting** [17], we count rendering triangles from `point_cloud_state_dict.pt`. Although both measure rendering load, #G and #T are not directly interchangeable and are therefore reported in separate rows rather than a single merged ranking. All methods use the same COLMAP [40] sparse

point cloud initialization and 30,000 training iterations on the 11 ScanNet++ [57] iPhone scenes. Table 3 summarizes rendering primitive counts and storage. Table 4 reports the explicit plane and superquadric blocks of Primimix in addition to its rendering Gaussians.

Table 3: Rendering compactness on ScanNet++ [57] iPhone dataset (11-scene average). #G: Gaussian surfels; #T: rendering triangles (MeshSplatting [17] only). Storage refers to the exported rendering asset (`point_cloud.ply` or `point_cloud_state_dict.pt`).

Method	Primitive	Count ↓	Storage (MB) ↓
2D Gaussian Splatting [19]	#G	**1.398M**	<u>325.4</u>
3D Gaussian Splatting [22]	#G	2.224M	526.1
MeshSplatting [17]	#T	1.654M	**188.3**
Primimix (Ours)	#G	<u>1.629M</u>	385.4

Table 4: Structural compactness of Primimix on ScanNet++ [57] iPhone dataset(11-scene average). Explicit blocks (#Plane, #SQ) capture coarse geometry; #Total counts the total number of two primitive types; #G counts the Gaussian skin used for final rendering.

	#Plane	#SQ	#Total	#G
Primimix (Ours)	535	87	622	1.629M

From Table 3, 3D Gaussian Splatting and 2D Gaussian Splatting require 2.224 M and 1.398 M rendering Gaussians (526.1 MB and 325.4 MB, respectively), without any explicit scene structure. MeshSplatting is the most storage-efficient (188.3 MB) and represents the scene with 1.654 M triangles, but at the cost of lower NVS quality (Table 2). Primimix uses 1.629 M Gaussians and 385.4 MB for rendering: fewer Gaussians than 3D Gaussian Splatting (∼27% reduction), but slightly more than 2D Gaussian Splatting. Thus, rendering compactness alone does not fully capture our advantage. The structural benefit becomes clear in Table 4: on average, only 535 plane blocks and 87 superquadric blocks (622 in total) explain the coarse scene geometry, while 1.629 M Gaussians serve primarily as a coupled appearance skin for final rasterization. Compared with millions of independent surfels or triangles, this separates *interpretable geometric structure* (hundreds of blocks) from *dense appearance parameterization* (millions of Gaussians). In other words, Primimix trades a modest reduction in rendering primitive count for an explicit part-level representation that Gaussian-only and mesh-only baselines do not provide.

4.3 Qualitative Results

We organize the qualitative evaluation into three figures, each aligned with one experimental axis in Secs. 4.2.1–4.2.3. Figure 6 inspects *geometric reconstruction*; Figure 7 compares *rendering quality* on held-out views; and Figure 8 illustrates *representation structure*, which provides the visual counterpart to the compactness analysis in Table 3.

Geometry Reconstruction (Fig. 6). We compare exported evaluation meshes from the same viewpoint against the aligned ground-truth scan (`mesh_aligned_0.05.ply`). For

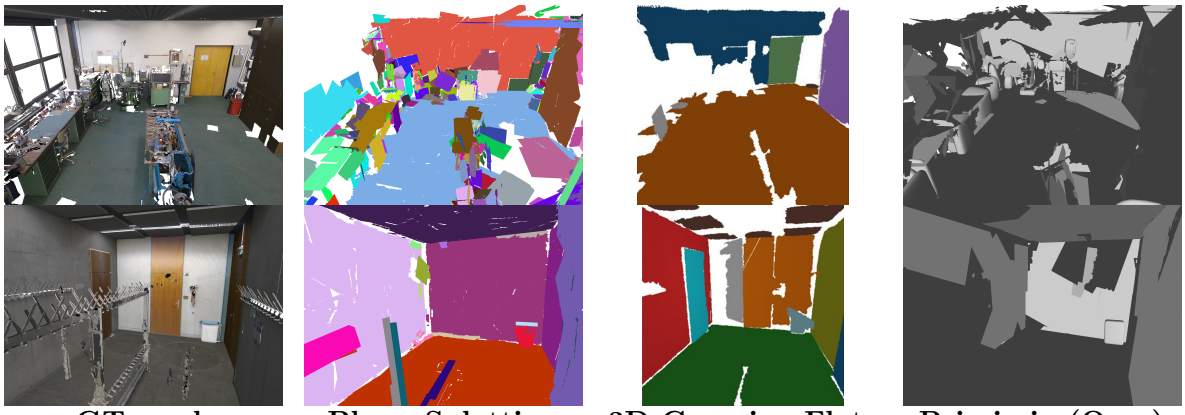


Figure 6: **Qualitative geometry comparison** on two ScanNet++ [57] scenes. Top row: `0e75f3c4d9`; bottom row: `7079b59642`. Each row shows the same view for one scene; columns compare the ground-truth mesh and the geometry mesh from PlanarSplatting [44], 3D Gaussian Flats [43] and Primimix.

Primimix, this mesh is the PlanarSplatting-style merged `eval_mesh.ply` (planar regions + superquadric geometry), i.e., the same asset used for the quantitative metrics in Table 1. PlanarSplatting [44] recovers clean wall and floor structure but often omits compact furniture. 3D Gaussian Flats [43] leaves visible gaps in large planar regions, consistent with its high completeness error in Table 1. Primimix combines architectural planes with object-level superquadrics, yielding more complete room structure on challenging layouts (top row: `0e75f3c4d9`) while avoiding the large missing regions seen in 3D Gaussian Flats; on the second scene (bottom row: `7079b59642`), all methods recover the global layout, with Primimix slightly better preserving furniture-scale geometry than the plane-only baseline.

Novel view synthesis (Fig. 7). We show rendered test views from 3D Gaussian Splatting [22], 2D Gaussian Splatting [19], and MeshSplatting [17] alongside the ground-truth image on the same `llffhold=8` frame. All baselines are trained on identical iPhone streams initialized from COLMAP [40] sparse point cloud and produce plausible RGB appearance, with 3D Gaussian Splatting generally achieving the sharpest textures. MeshSplatting yields connected mesh-based renderings with competitive detail, while 2D Gaussian Splatting produces slightly softer results in textured regions. Primimix does not match 3D Gaussian Splatting pixel-wise, so this might be a reason why it fails to render the iron rack in the scene `0e75f3c4d9`. However, it remains almost in the same perceptual ballpark on several scenes, illustrating that structured reconstruction can still produce plausible novel views.

Representation structure (Fig. 8). Compactness is difficult to convey with a single RGB render, since the advantage of Primimix lies in *how* the scene is encoded rather than in a different camera view. We therefore show a representative scene, `0a7cc12c0e`, where the explicit plane/superquadric decomposition is visually clear. On this scene, Primimix uses ∼626 explicit blocks (∼525 planes + ∼101 superquadrics) together with ∼1.324 M surface Gaussians after Stage II, whereas 3D Gaussian Splatting [22] stores ∼1.422 M

Figure 7: **Qualitative novel view synthesis** on held-out test frames from two ScanNet++ [57] iPhone scenes. Top row: `0e75f3c4d9`; bottom row: `7079b59642`. Each row shows the same test view for one scene; columns compare the ground-truth image and renders from 2D Gaussian Splatting (2DGS) [19], 3D Gaussian Splatting (3DGS) [22], MeshSplatting [17], and Primimix.

Figure 8: **Representation structure** on ScanNet++ [57] iPhone scene `0a7cc12c0e`. **Left:** Stage-I explicit planar and superquadric blocks. **Middle:** Stage-II render. **Right:** subsampled 3DGS Gaussian centers from the exported `point_cloud.ply`, illustrating the dense unstructured splat field required by the baseline (Table 3).

unstructured Gaussians in `point_cloud.ply`.

Left: Stage-I primitive decomposition rendered from `renders_primitives` (colored planar and superquadric blocks).

Middle: the corresponding Stage-II shaded render at the same camera.

Right: subsampled 3DGS Gaussian centers from the baseline `point_cloud.ply`, illustrating the dense splat field without part-level organization.

The contrast shows that Primimix explains the room with a small set of interpretable geometric primitives and a moderate number of surface Gaussians, whereas 3D Gaussian Splatting distributes millions of kernels throughout the volume without an explicit part structure.

5 Discussion

5.1 Preprocessing Challenges

Reconstructing a single, denoised 3D point cloud with globally consistent instance labels from multi-view images was the most difficult part of our preprocessing. Lifting per-view 2D segmentation into 3D fragmented and duplicated each object across views, and reconciling these per-view instances by cross-frame association required many scene-specific heuristics that did not generalize. Monocular depth, in turn, is noisy and inconsistent between views. We ultimately chose the MoGe–TSDF densification and LaSSM segmentation of Sec. 3.2.2–3.2.3. The earlier exploratory approaches and the reasons we abandoned them are documented in the Appendix.

5.2 Challenges with Training

Jointly optimizing geometry and appearance while balancing planar and volumetric regions is a challenging problem. Particularly, superquadrics are extremely hard to optimize, and little research exists on their application to scene-level tasks. During development, we observed many failure cases where planes intruded into objects better represented with superquadrics, like wardrobes or chairs. On other cases, it is hard to judge whether planes or superquadrics better fit an object, like with whiteboards or tables. Calibrating the number of gaussians needed for the skinning, or the number of primitives to represent the scene itself was also difficult. Ultimately, choices like using SuperDec as a strong initialization prior and adding the primitive regularizers presented in Section 3.4.1 made it possible for the superquadrics to converge and make both primitives stay away from each other in most cases.

5.3 Limitations

Our pipeline inherits the quality of its upstream components. The geometric scaffold is initialized from MoGe-2 depth and normal predictions and from LaSSM instance segmentation, so errors in either propagate into the primitive fit. LaSSM is trained on laser-scan meshes but applied to the noisier MoGe–TSDF reconstruction (Sec. 3.2.3); under this domain gap the instance structure suffices to separate compact objects from planar surfaces, but segmentation errors are not corrected downstream.

The initialization priors also bound what the representation can express. SuperDec seeding caps each volumetric instance at eight superquadrics and falls back to a single ellipsoid when an instance is too small (Sec. 3.3.3), which limits the detail recoverable for complex objects. The geometric scaffold $\mathcal{S}$ is frozen during the photometric phase (Sec. 3.4.2); once the geometric phase has converged, the primitive parameters no longer adapt to the appearance cues that drive Gaussian refinement.

Finally, our evaluation is confined to indoor ScanNet++ [57] iPhone scenes, so generalization to outdoor or non-room-scale environments remains untested.

6 Conclusion

We presented Primimix, the first hybrid full indoor scene decomposition and rendering approach that utilizes three complementary parametric primitives: bounded planes for flat structural backgrounds, superquadrics for curved or irregular foreground objects, and 2D Gaussians for smooth surfaces and realistic rendering.

On ScanNet++ [57], Primimix improves geometry reconstruction over plane-based baselines, with mean Chamfer distance **6.63 cm**, accuracy **7.00 cm**, completeness **6.26 cm**, and F@5 cm **62.80%**, compared to PlanarSplatting [44] (10.99 cm / 9.58 cm / 12.41 cm / 44.70%) and 3D Gaussian Flats [43] (18.60 cm / 4.60 cm / 32.59 cm / 50.24%). Novel view synthesis remains below unstructured 3D Gaussian Splatting [22] and 2D Gaussian Splatting [19] (PSNR **18.74** vs. 19.62 / 19.22; SSIM **0.8206** vs. 0.8627 / 0.8445; LPIPS **0.2936** vs. 0.2269 / 0.2663), but is on par with MeshSplatting [17] in PSNR while achieving clearly better SSIM and LPIPS (0.8206 / 0.2936 vs. 0.7845 / 0.3776). In compactness, Primimix averages **622** explicit blocks (**535** planes + **87** superquadrics) and **1.629 M** surface Gaussians (**385.4 MB**), reducing Gaussian count by ∼27% relative to 3D Gaussian Splatting (**2.224 M** / **526.1 MB**) while exposing an interpretable geometric scaffold that neither unstructured splatting nor plane-only reconstruction provides. Thus, the hybrid design separates a compact, part-level geometric core from a moderate Gaussian skin used for high-fidelity rendering, trading a modest novel view synthesis gap to unstructured baselines for stronger geometry and structured scene encoding.

Nevertheless, several limitations are present as outlined in Sec. 5.3. Future work includes refining instance labels jointly with the primitives or by closing the LaSSM domain gap, which could make the initialization more robust and generalizable. Secondly, relaxing the frozen scaffold to permit bounded geometric adjustment during the second photometric phase could improve the geometric quality. Finally, further evaluating our approach on other indoor and outdoor datasets would give more insights into the generalizability of our approach.

References

[1] Alan H. Barr. "Superquadrics and Angle-Preserving Transformations". In: *IEEE Computer Graphics and Applications* 1.1 (1981), pp. 11–23.

[2] Shariq Farooq Bhat, Ibraheem Alhashim, and Peter Wonka. "AdaBins: Depth Estimation Using Adaptive Bins". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2021.

[3] Shariq Farooq Bhat et al. "ZoeDepth: Zero-Shot Transfer by Combining Relative and Metric Depth". In: *arXiv preprint arXiv:2302.12288* (2023).

[4] Nicolas Carion et al. "SAM 3: Segment Anything with Concepts". In: *International Conference on Learning Representations (ICLR)*. 2026.

[5] Zhaiyu Chen et al. "Parametric Point Cloud Completion for Polygonal Surface Reconstruction". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2025.

[6] Brian Curless and Marc Levoy. "A Volumetric Method for Building Complex Models from Range Images". In: *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*. ACM, 1996, pp. 303–312.

[7] Angela Dai et al. "ScanNet: Richly-annotated 3D Reconstructions of Indoor Scenes". In: *Proc. Computer Vision and Pattern Recognition (CVPR), IEEE*. 2017.

[8] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, et al. "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale". In: *International Conference on Learning Representations*. 2021.

[9] Alexey Dosovitskiy et al. "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale". In: *International Conference on Learning Representations (ICLR)*. 2021.

[10] David Eigen, Christian Puhrsch, and Rob Fergus. "Depth Map Prediction from a Single Image Using a Multi-Scale Deep Network". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2014.

[11] Elisabetta Fedele et al. "SuperDec: 3D Scene Decomposition with Superquadric Primitives". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2025.

[12] Pedro F. Felzenszwalb and Daniel P. Huttenlocher. "Efficient Graph-Based Image Segmentation". In: *International Journal of Computer Vision (IJCV)* 59.2 (2004), pp. 167–181.

[13] Martin A. Fischler and Robert C. Bolles. "Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography". In: *Commun. ACM* 24.6 (June 1981), pp. 381–395. ISSN: 0001-0782. DOI: 10.1145/358669.358692. URL: https://doi.org/10.1145/358669.358692.

[14] Huan Fu et al. "Deep Ordinal Regression Network for Monocular Depth Estimation". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018.

[15] Zhirui Gao et al. "Self-supervised Learning of Hybrid Part-aware 3D Representations of 2D Gaussians and Superquadrics". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. Oct. 2025, pp. 9649–9659.

[16] Antoine Guédon and Vincent Lepetit. "SuGaR: Surface-Aligned Gaussian Splatting for Efficient 3D Mesh Reconstruction and High-Quality Mesh Rendering". In: *Pro-*

ceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2024, pp. 5354–5363. DOI: 10.1109/CVPR52733.2024.00512.

[17] Jan Held et al. "MeshSplatting: Differentiable Rendering with Opaque Meshes". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2026, pp. 7320–7329.

[18] Jan Held et al. "Triangle Splatting for Real-Time Radiance Field Rendering". In: *Proceedings of the International Conference on 3D Vision (3DV)*. 2026.

[19] Binbin Huang et al. "2D Gaussian Splatting for Geometrically Accurate Radiance Fields". In: *ACM SIGGRAPH 2024 Conference Papers*. 2024. DOI: 10.1145/3641519.3657428.

[20] Xirui Jin et al. "PlanarGS: High-Fidelity Indoor 3D Gaussian Splatting Guided by Vision-Language Planar Priors". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2025.

[21] Ian T. Jolliffe. *Principal Component Analysis*. 2nd ed. Springer Series in Statistics. Springer, 2002.

[22] Bernhard Kerbl et al. "3D Gaussian Splatting for Real-Time Radiance Field Rendering". In: *ACM Transactions on Graphics* 42.4 (July 2023). URL: https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/.

[23] Alexander Kirillov et al. "Segment Anything". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2023.

[24] Maxim Kolodiazhnyi et al. "OneFormer3D: One Transformer for Unified Point Cloud Segmentation". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2024.

[25] Chen Liu et al. "PlaneRCNN: 3D Plane Detection and Reconstruction from a Single Image". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019.

[26] Weixiao Liu et al. "Robust and Accurate Superquadric Recovery: A Probabilistic Approach". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2022.

[27] Daniel MacSwayne, Ales Leonardis, and Jianbo Jiao. "3D Superquadric Splatting". In: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. 2026, pp. 5154–5163.

[28] Ben Mildenhall et al. "NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis". In: *European Conference on Computer Vision (ECCV)*. 2020.

[29] Tom Monnier et al. "Differentiable Blocks World: Qualitative 3D Decomposition by Rendering Primitives". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023.

[30] Richard A. Newcombe et al. "KinectFusion: Real-Time Dense Surface Mapping and Tracking". In: *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*. 2011, pp. 127–136.

[31] Maxime Oquab et al. "DINOv2: Learning Robust Visual Features without Supervision". In: *Transactions on Machine Learning Research (TMLR)* (2024).

[32] Despoina Paschalidou, Ali Osman Ulusoy, and Andreas Geiger. "Superquadrics Revisited: Learning 3D Shape Parsing Beyond Cuboids". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019.

[33] Luigi Piccinelli et al. "UniDepth: Universal Monocular Metric Depth Estimation". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2024.

[34] Charles R. Qi et al. "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017.

[35] Charles R. Qi et al. "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017.

[36] René Ranftl, Alexey Bochkovskiy, and Vladlen Koltun. "Vision Transformers for Dense Prediction". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2021.

[37] René Ranftl et al. "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 44.3 (2022), pp. 1623–1637.

[38] Nikhila Ravi et al. "SAM 2: Segment Anything in Images and Videos". In: *International Conference on Learning Representations (ICLR)*. 2025.

[39] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. "U-Net: Convolutional Networks for Biomedical Image Segmentation". In: *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*. 2015.

[40] Johannes Lutz Schönberger and Jan-Michael Frahm. "Structure-from-Motion Revisited". In: *Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016.

[41] Johannes Lutz Schönberger et al. "Pixelwise View Selection for Unstructured Multi-View Stereo". In: *European Conference on Computer Vision (ECCV)*. 2016.

[42] Jiahao Sun et al. "Superpoint Transformer for 3D Scene Instance Segmentation". In: *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 2023.

[43] Maria Taktasheva et al. "3D Gaussian Flats: Hybrid 2D/3D Photometric Scene Reconstruction". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2025. URL: https://openreview.net/forum?id=uVxQEIgXfL.

[44] Bin Tan et al. "PlanarSplatting: Accurate Planar Surface Reconstruction in 3 Minutes". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2025.

[45] Gabriel Tavernini et al. "SuperFlex: Deformable Superquadrics for Point Cloud Decomposition". In: *European Conference on Computer Vision (ECCV)*. 2026.

[46] Shubham Tulsiani et al. "Learning Shape Abstractions by Assembling Volumetric Primitives". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017.

[47] E.R. Van Dop and P.P.L. Regtien. "Fitting undeformed superquadrics to range data: improving model recovery and classification". In: *Proceedings. 1998 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (Cat. No.98CB36231)*. 1998, pp. 396–401. DOI: 10.1109/CVPR.1998.698636.

[48] Joanna Waczyńska et al. "Games: Mesh-based adapting and modification of gaussian splatting". In: *arXiv preprint arXiv:2402.01459* (2024).

[49] Ruicheng Wang et al. “MoGe-2: Accurate Monocular Geometry with Metric Scale and Sharp Details”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2025. URL: https://openreview.net/forum?id=16mDq7m2OK.

[50] Ruicheng Wang et al. “Moge: Unlocking accurate monocular geometry estimation for open-domain images with optimal training supervision”. In: *Proceedings of the Computer Vision and Pattern Recognition Conference*. 2025, pp. 5261–5271.

[51] Shuzhe Wang et al. “DUSt3R: Geometric 3D Vision Made Easy”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2024.

[52] Enze Xie et al. “SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2021.

[53] Mutian Xu et al. “SAMPro3D: Locating SAM Prompts in 3D for Zero-Shot Instance Segmentation”. In: *International Conference on 3D Vision (3DV)*. 2025.

[54] Lihe Yang et al. “Depth Anything V2”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2024.

[55] Lihe Yang et al. “Depth Anything: Unleashing the Power of Large-Scale Unlabeled Data”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2024.

[56] Lei Yao et al. “LaSSM: Efficient Semantic-Spatial Query Decoding via Local Aggregation and State Space Models for 3D Instance Segmentation”. In: *IEEE Transactions on Circuits and Systems for Video Technology (TCSVT)* (2026).

[57] Chandan Yeshwanth et al. “ScanNet++: A High-Fidelity Dataset of 3D Indoor Scenes”. In: *Proceedings of the International Conference on Computer Vision (ICCV)*. 2023.

[58] Kadir Yilmaz et al. “Volume Transformer: Revisiting Vanilla Transformers for 3D Scene Understanding”. In: *European Conference on Computer Vision (ECCV)*. 2026.

[59] Wei Yin et al. “Learning to Recover 3D Scene Shape from a Single Image”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2021.

[60] Wei Yin et al. “Metric3D: Towards Zero-Shot Metric 3D Prediction from a Single Image”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2023.

Appendix

Exploratory Approaches for Panoptic Point Cloud Initialization

Before adopting the MoGe–TSDF densification and LaSSM segmentation of the main text (Sec. 3.2.2–3.2.3), we explored several image-based approaches to building a panoptic point cloud. We record them here for completeness.

2D Segmentation and Lifting to 3D

As an initial approach, we investigated whether image-level segmentation could be used to construct a panoptic point cloud. The main idea was to first obtain object and surface masks independently in each image, and then lift these masks into the 3D scene using depth and camera information. This would allow us to convert 2D segmentation results into point groups that could be used for primitive initialization.

For each image, a segmentation model was prompted with object-like and planar categories. Object-like categories were intended to identify regions such as chairs, tables, cabinets, monitors, and other foreground objects. Planar categories were intended to identify large surface regions such as walls, floors, ceilings, doors, windows, and tabletops. The segmentation model itself did not determine the corresponding primitive representation. Instead, each semantic prompt was manually assigned to either the superquadric or planar primitive family. This assignment therefore represents a semantic prior about the intended scene representation rather than a geometric verification that an individual region is planar or superquadric-shaped. This distinction was motivated by the structure of our scene representation, where object-like regions are better represented by superquadric primitives, while large flat regions are better represented by planar primitives.

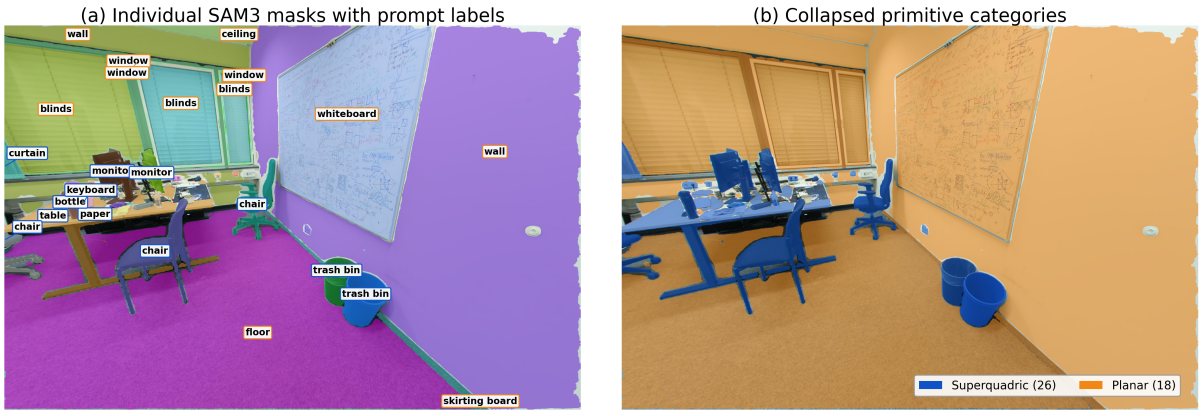


Figure 9: Example of SAM3 labels and their assignment to primitive families. The left image shows the masks and semantic classes predicted from the specified prompt vocabulary. The right image shows the manually defined mapping of these semantic classes to the superquadric and planar primitive families; for example, the class *chair* is assigned to the superquadric family.

After obtaining the 2D masks, each labeled pixel was projected into 3D using the camera

backprojection formula

$$\mathbf{x}_c = d(u,v)\,\mathbf{K}^{-1}\begin{bmatrix} u \\ v \\ 1 \end{bmatrix}, \qquad \mathbf{x}_w = \mathbf{R}_{wc}\,\mathbf{x}_c + \mathbf{t}_{wc}, \tag{19}$$

where (u, v) denotes the pixel coordinates, $d(u, v)$ is the corresponding depth value, and $\mathbf{K}$ is the camera intrinsic matrix. The resulting point $\mathbf{x}_c \in \mathbb{R}^3$ is expressed in the camera coordinate system and is transformed into world coordinates using the camera-to-world rotation $\mathbf{R}_{wc}$ and translation $\mathbf{t}_{wc}$. This operation was applied to pixels that belonged to a predicted mask and had a valid corresponding depth value. Each lifted point inherited the semantic prompt, primitive category, and per-image mask identity of its source pixel. For computational and storage efficiency, the resulting point sets were subsampled when the number of valid pixels in a mask exceeded the configured per-mask limit.

This allowed us to obtain separate 3D point sets for object-like and planar regions without requiring pixel-wise manual annotation of the scene. An output example can be seen in figure 10. However, the prompt vocabulary and the mapping from semantic prompts to primitive families were specified manually. In principle, these point sets contained the geometric and semantic information required to initialize superquadric primitives from object regions and planar primitives from large surface regions.

Although the lifting produced a geometrically interpretable result, however, the lifted masks retained their per-image instance identities. Consequently, observations of the same physical object from different camera views were represented as separate point sets with different instance IDs. Directly using these point sets for primitive initialization would therefore produce multiple fragmented or overlapping primitives for the same physical object. Partial masks could additionally initialize primitives representing only small visible parts of an object. Another downside of this procedure is that it transferred errors from both the segmentation and depth estimation stages into the point cloud.

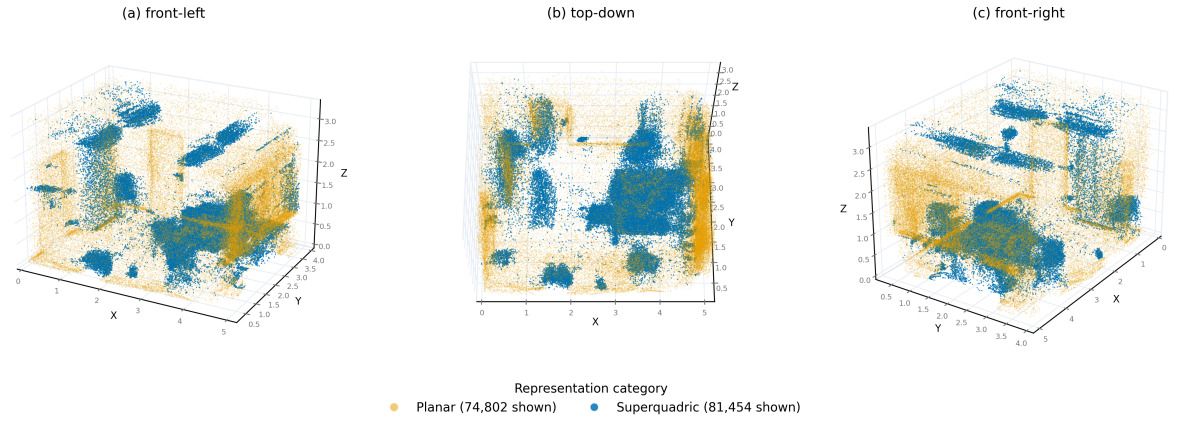


Figure 10: Example of the lifted SAM3 segmentations, with planar points shown in orange and superquadric candidate points shown in blue. The visualization contains accumulated observations from multiple camera views before cross-frame instance association.

Cross-Frame Instance Association

To address the fragmentation and obtain scene-level object identities, we explored a graph-based association strategy across views. Each lifted mask assigned to the superquadric category was treated as a node in a graph. Candidate edges are created between instances from different images when their 3D point sets are spatially close and when they are their mask has the same predicted class label. Formally, two nodes (i) and (j) were considered as candidates only if they originated from different images, shared the same prompt, and satisfied

$$|\boldsymbol{\mu}_i - \boldsymbol{\mu}_j|_2 \leq 1.25\text{m},$$

where ($\boldsymbol{\mu}_i$) and ($\boldsymbol{\mu}_j$) are the centroids of their lifted point sets. Instances containing fewer than (20) points were ignored, and at most the (50) nearest candidate nodes were evaluated for each instance. For example, a mask predicted as *chair* was compared with other *chair* masks but not with masks predicted as *table*. This restriction reduced associations between semantically different objects, but it also prevented valid associations when the same object received different prompts in different views. To evaluate whether two candidate nodes should be connected, a subsampled set of points from one lifted instance was projected into the camera view of the other mask. Only points that projected into the image and were sufficiently consistent with the observed depth map were considered visible. The fraction of these visible points falling inside the target mask provided a directed overlap score. For a candidate pair of instances i and j, the directed overlap score was defined as

$$o_{i\to j} = \frac{|\{\mathbf{p} \in V_{i\to j} \,|\, \pi_j(\mathbf{p}) \in M_j\}|}{|V_{i\to j}|}, \tag{20}$$

where $V_{i\to j}$ denotes the set of sampled points from instance i that are visible and depth-consistent in view j, π_j denotes projection into view j, and M_j is the target mask. Up to 512 points were sampled from each instance, and at least 30 valid projected points were required. Depth consistency was evaluated using an absolute tolerance of 0.10 m and a relative tolerance of 4%.

The bidirectional overlap score was defined as

$$o_{ij}^{\text{bi}} = \min\left\{o_{i\to j}, o_{j\to i}\right\}. \tag{21}$$

An association was accepted when

$$o_{ij}^{\text{bi}} \geq 0.35. \tag{22}$$

The test was then applied in both directions, and an association was accepted only when the bidirectional overlap exceeded a specified threshold. This reduced associations caused by accidental overlap in a single viewing direction. Accepted pairwise associations were merged using a union-find data structure. All masks belonging to the same resulting connected component were assigned a shared global instance ID.

The merged components were subsequently filtered according to their global support across source masks and camera views. Components that were observed in too few masks or images were treated as unsupported. These additional criteria were intended to remove accidental or weakly supported detections but introduced further scene-dependent thresholds. In the tested configuration, a merged component was retained only if it was

supported by at least 50 source masks from at least 50 distinct images. The parameter values were selected empirically by repeatedly running the association on representative scenes and visually inspecting the accepted graph edges and the resulting point cloud. The thresholds were adjusted to reduce clearly incorrect merges while preserving observations that appeared to belong to the same physical object. This iterative visual tuning also showed that stricter values removed valid partial observations, whereas more permissive values frequently merged nearby objects of the same class.

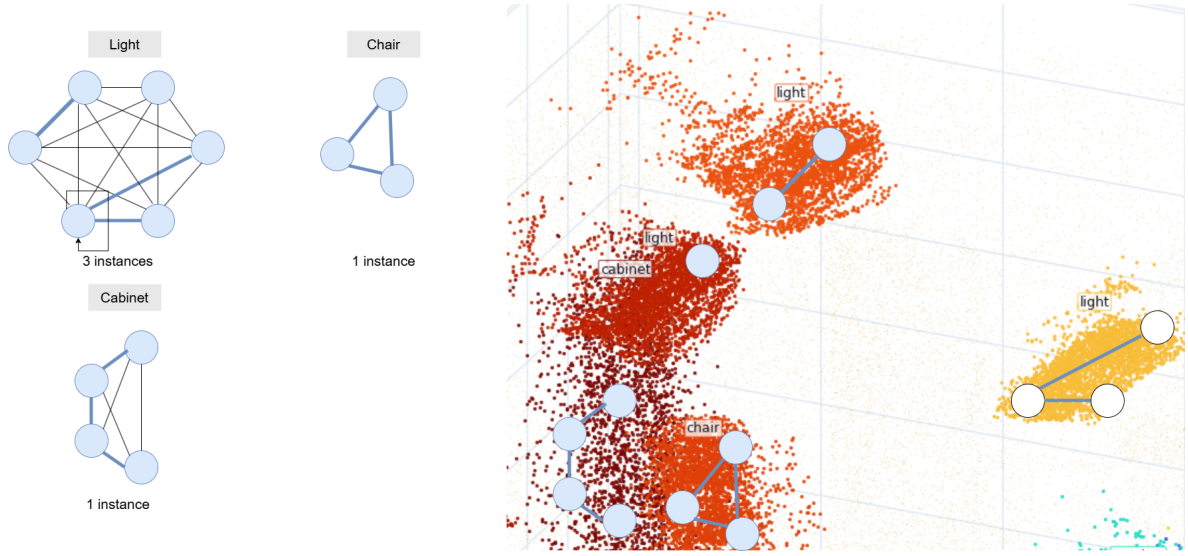


Figure 11: Visualization of the graph-based cross-frame instance association. Each node represents a lifted mask instance, and graphs are constructed separately for masks sharing the same semantic prompt. Thin black edges indicate candidate associations between instances, while thick blue edges indicate pairs accepted after distance evaluation of centroids, bidirectional projection, mask-overlap, and depth-consistency verification. Connected components of accepted edges define the global instance IDs.

This graph-based formulation directly attempted to convert many per-view mask IDs into fewer scene-level object instances. However, the method introduced several additional rules and thresholds including spatial distance thresholds, projected-mask overlap, depth consistency tolerances, and minimum point counts. As a result, the consistency fix became increasingly dependent on heuristic parameters whose optimization is challenging and that increase complexity. While useful as an exploratory step, this made the approach less attractive as the final initialization pipeline.

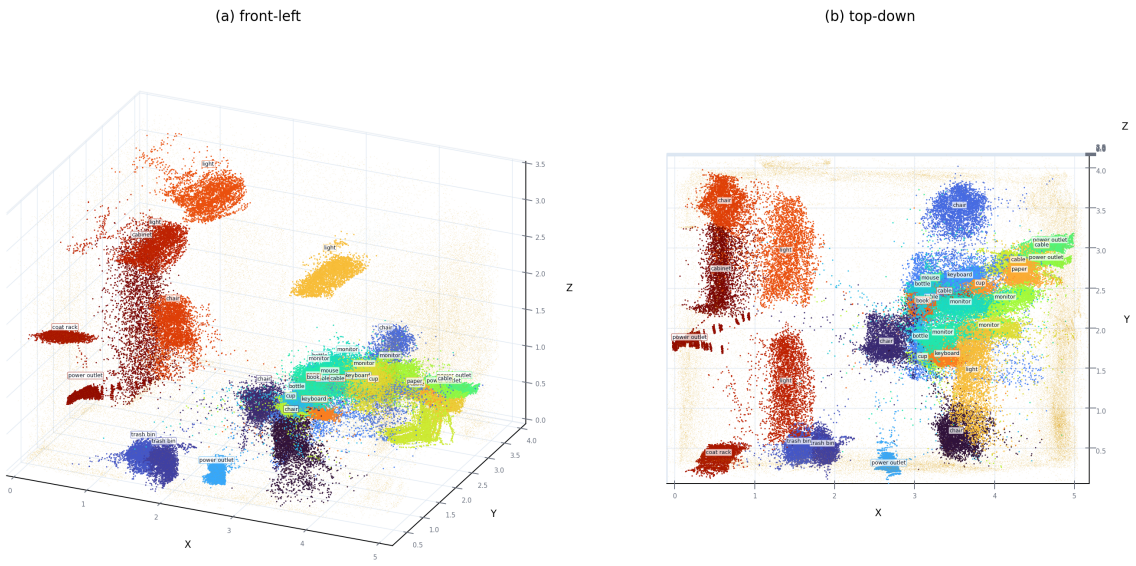


Figure 12: Point cloud obtained after graph-based cross-frame instance association. Colors denote the global instance IDs produced by the union-find connected components, while the text annotations show their semantic prompts. Masks from different camera views that were accepted as observations of the same object are assigned to the same global component.

SAM3 Video Segmentation

The association stage of the previous section attempted to repair inconsistent per-view identities after they had been produced, and the number of thresholds it required suggested that the reconciliation itself was the difficulty. We therefore turned to the video mode of SAM3 [4], reviewed in Sec. 2.4, which offered to avoid the inconsistency rather than repair it. Prompted with a set of object and surface categories, SAM3 detects the prompted concepts on each frame and tracks each detected instance through the sequence as a *masklet*, a temporal segment carrying a single persistent identity. If each physical object is covered by one masklet across the whole sequence, the points lifted from all of its views inherit one instance identity by construction, without the per-view duplication that the association stage had to resolve. We treated the frames of our test scene as a video, prompted the model with the categories of the first approach (45 concepts in total, 16 planar and 29 object-like), and lifted the resulting label maps into 3D exactly as before, assigning each lifted point the masklet identity of its pixel.

Applying the tracker to the full 369-frame sequence in a single pass proved too slow to be practical. SAM3 maintains the memory of each masklet independently and attends to every active masklet on each frame, so its per-frame cost increases with the number of masklets held. A decomposition of an entire room prompted with 45 concepts produces many of these from the outset. 37 were tracked in the first frame alone. And their number grows further as the camera reaches new parts of the scene. The per-frame processing time accordingly rose steeply, from roughly two seconds at the start to more than twenty-five minutes by frame 95, which extrapolates to a runtime of several days on an RTX 5090,

and the pass did not complete.

To make the tracker usable, we processed the sequence in overlapping windows of 60 frames, each tracked as an independent session whose memory was released before the next, with consecutive windows sharing 15 frames. Masklet identities are consistent within a window but independent between windows, and the shared frames provide the correspondence needed to relate them, since both sessions segment the same objects in the same shared images and their masks there nearly coincide. We matched masks of the same category across the two sessions by their intersection over union (IoU), defined as the area in which the two masks overlap divided by the area they jointly cover, and therefore equal to one for identical masks and zero for disjoint ones; we accepted a match when this value exceeded 0.3 and merged the matched identities transitively across all window boundaries. This kept the per-frame cost constant, reduced the runtime for the full scene to approximately 21 minutes, and yielded 223 globally consistent identities, in place of the 13,729 unassociated per-view instances the per-image pipeline produced for the same scene.

Making the tracker fast enough, however, did not make it reliable, and the reason is intrinsic to the data. Splitting the sequence means that identities must be reconciled across each window boundary, and this reconciliation is only as reliable as the agreement between the masks in the frames the two windows share. For a continuous video these shared frames would be almost identical and their masks would agree closely; the ScanNet++ [57] frames, however, are not a continuous recording but a set of high-resolution photographs taken to cover the room from many angles, so even the shared frames observe an object from perceptibly different sides, and the reconciliation again rested on a hand-tuned overlap threshold. No single threshold suited both small objects, whose masks must nearly coincide to be matched, and large partially observed surfaces such as walls, whose masks in two windows differ even when they belong to the same surface; objects absent from the shared frames could not be reconciled at all and received two identities. Temporal tracking thus anchors instance identity in the order of the frames, which is informative only while consecutive frames remain coherent, whereas our frames are related to one another only through the 3D scene they observe. This led us to anchor instance identity in 3D directly.

SAMPro3D Cross-frame Instance Segmentation

SAMPro3D [53] anchors instance identity in 3D. In contrast to SAM3, it builds on the original SAM [23], reviewed in Sec. 2.4, whose prompts are geometric rather than textual: given a point, SAM returns a mask of the object beneath it, without any notion of object class. SAMPro3D places these prompts in 3D. It samples a fixed fraction of the scene points by farthest point sampling, which spreads the resulting *3D prompts* evenly enough to cover every instance, and projects each prompt into every frame using the camera pose and intrinsics; an occlusion test against the frame's depth map discards the projections that fall behind nearer geometry, and each remaining pixel is passed to SAM as a point prompt. Because the pixel prompts of one 3D prompt are all projections of the same physical point, the masks they generate refer to the same object in every frame by construction, so cross-frame consistency follows without any tracking or association,

and the method remains training-free and class-agnostic. Two refinements complete it: a view-guided filter that scores each prompt by the predicted quality and the stability of its masks across all frames in which it appears and discards the unreliable ones, and a consolidation step that merges prompts whose masks overlap, since a single prompt frequently segments only the part of an object that its frames happen to observe.

Because a 3D prompt projects independently into every frame that observes it, the ordering and spacing of the frames no longer affect the result, which is precisely the property that temporal tracking lacked. We ported the pipeline from its native ScanNet [7] RGB-D setting to our indoor scenes, sampling the 3D prompts from a scene point cloud and using per-view depth maps for the occlusion test, for which the visibility tolerance had to be relaxed relative to the original to accommodate our data. Every prompt yields masks that are consistent across all views by construction, so unlike the per-view instances of the first two approaches, each resulting instance is a single, view-stable segment rather than one identity per frame. On our test scene the pipeline produced 47 such instances, exported as a labelled point cloud together with the per-frame masks.

The number of instances, however, did not correspond to the number of objects, which is what primitive initialization requires. Because each 3D prompt segments only the surface visible in the frames that observe it, a large object is covered by several prompts, and the consolidation step is responsible for merging these into one instance. Where the prompts' masks do not overlap reliably across views this merging fails, so the object is left divided into several instances that are each individually view-consistent but collectively represent one physical object under different identities. Large planar structures such as walls and floors suffer most, since each frame captures only a portion of them; the original method already treats them as a difficult case and applies a dedicated post-processing step to the floor. Reducing this over-segmentation in our scenes required re-tuning the prompt sampling density together with the filtering and consolidation thresholds for every scene, and the settings that merged the pieces of a wall in one scene merged distinct furniture in another. Being class-agnostic, the segmentation moreover gave no signal for the later decision of which instances to represent by superquadrics and which by planes. These limitations motivated the approach adopted in the main text, a learned 3D instance segmentation model that produces instance and semantic labels jointly and requires no per-scene tuning (Sec. 3.2.3).